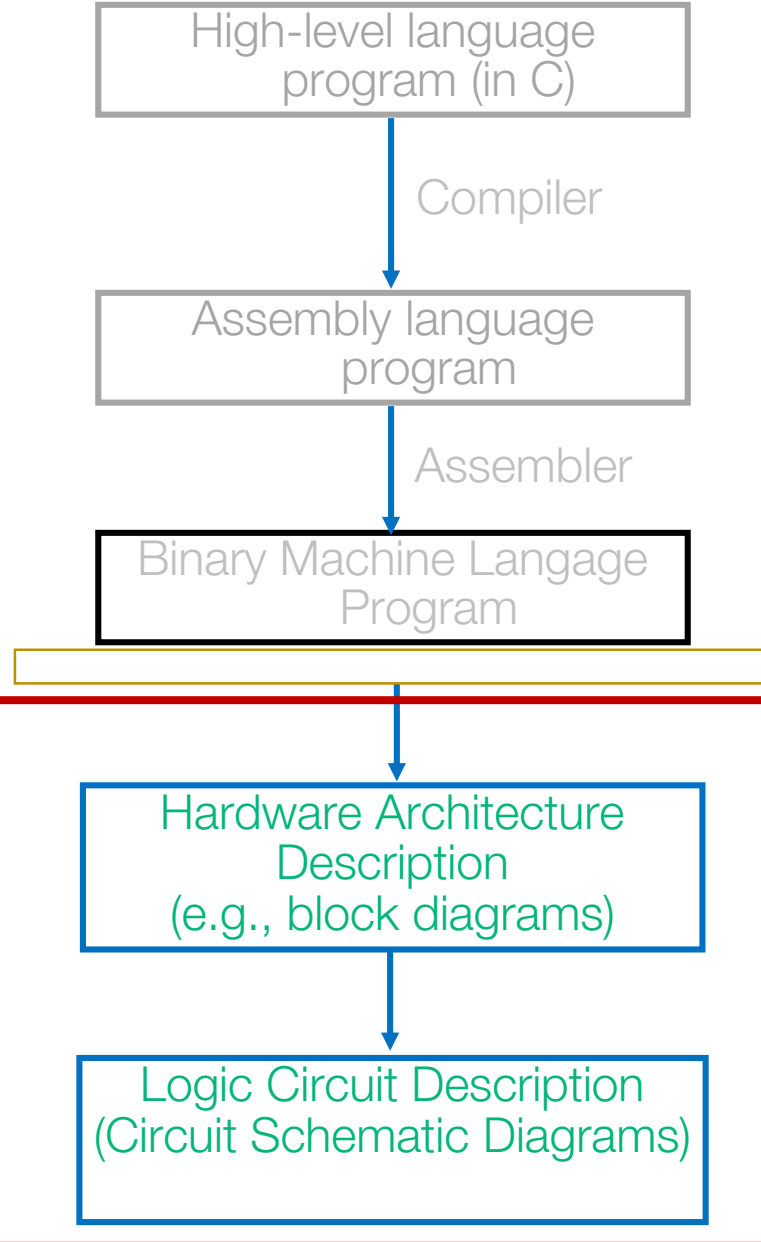


Computer Architecture II

RISC V Datapath

Building a RISC-V Processor

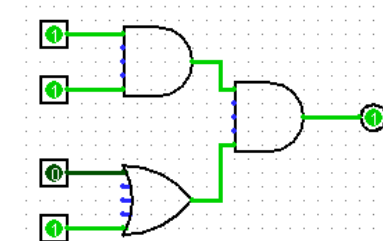
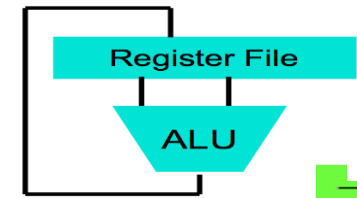
Levels of Transformation



```
temp = v[k];  
v[k] = v[k+1];  
v[k+1] = temp;
```

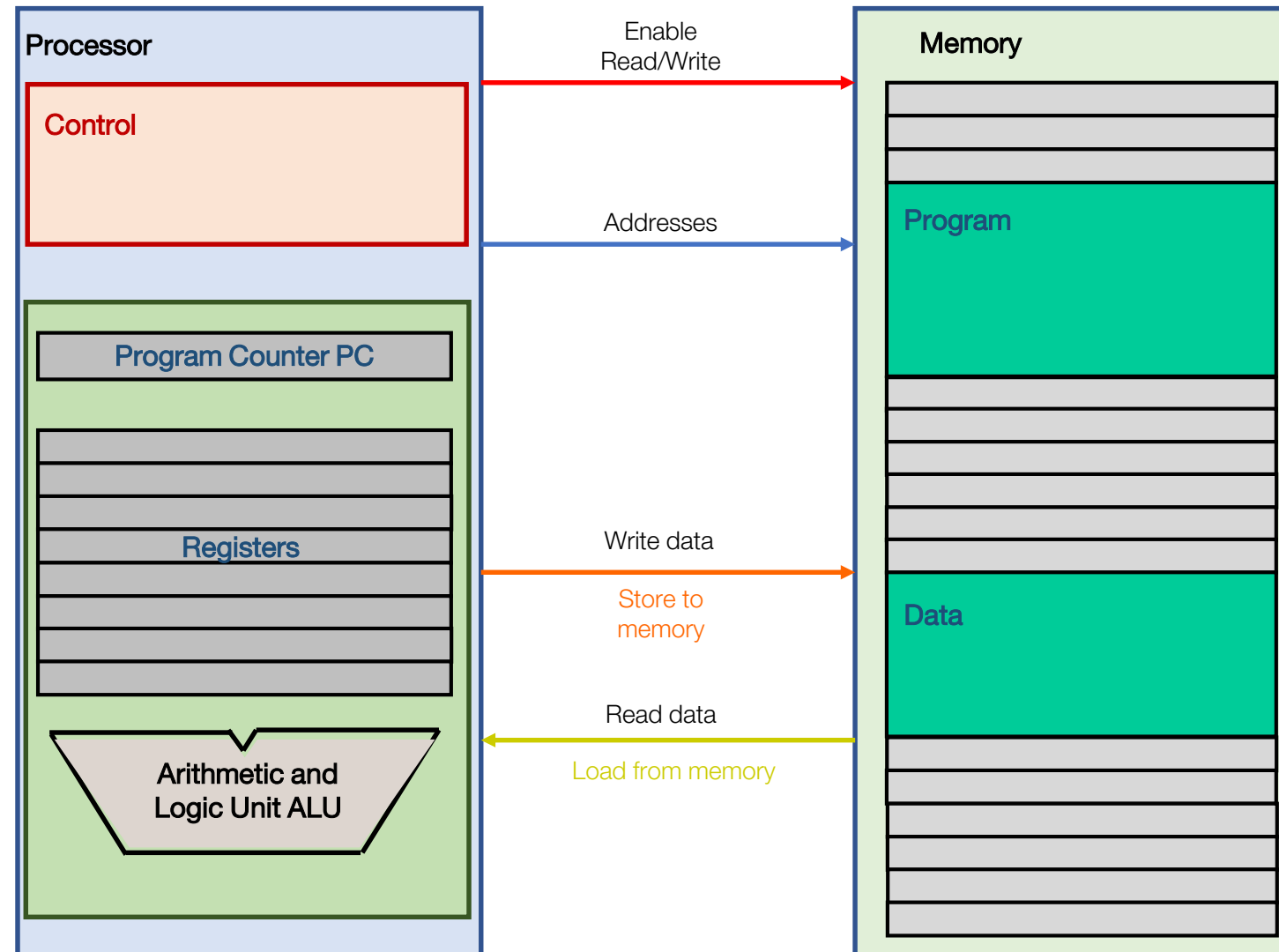
```
lw $15, 0($2)  
lw $16, 4($2)  
sw $16, 0($2)  
sw $15, 4($2)
```

```
0000 1001 1100 0110 1010 1111 0101 1000  
1010 1111 0101 1000 0000 1001 1100 0110  
1100 0110 1010 1111 0101 1000 0000 1001  
0101 1000 0000 1001 1100 0110 1010 1111
```



How do we build a Single-Core Processor?

- **Processor (CPU):** the active part of the computer that does all the work (data manipulation and decision-making).
- **Datapath (the brawn):**
 - portion of the processor that contains hardware necessary to perform operations required by the processor.
- **Control (the brain):**
 - portion of the processor (also in hardware) that tells the datapath what needs to be done.

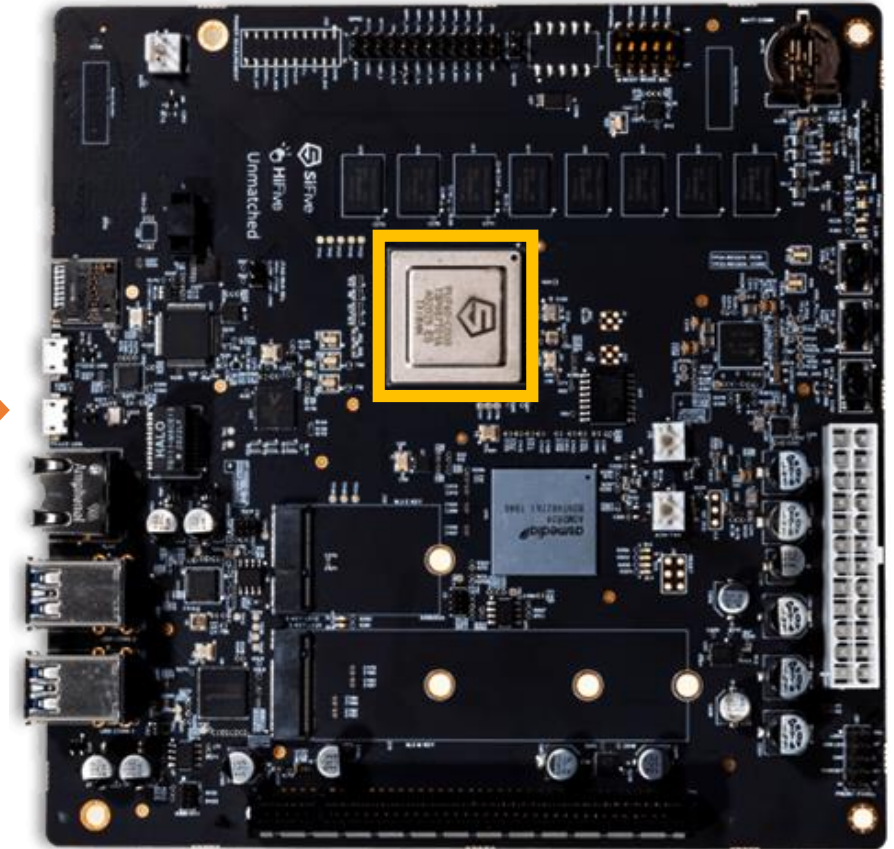


ensia How do we build a Single-Core Processor?

- Design Hardware to Execute All RV32I Instructions



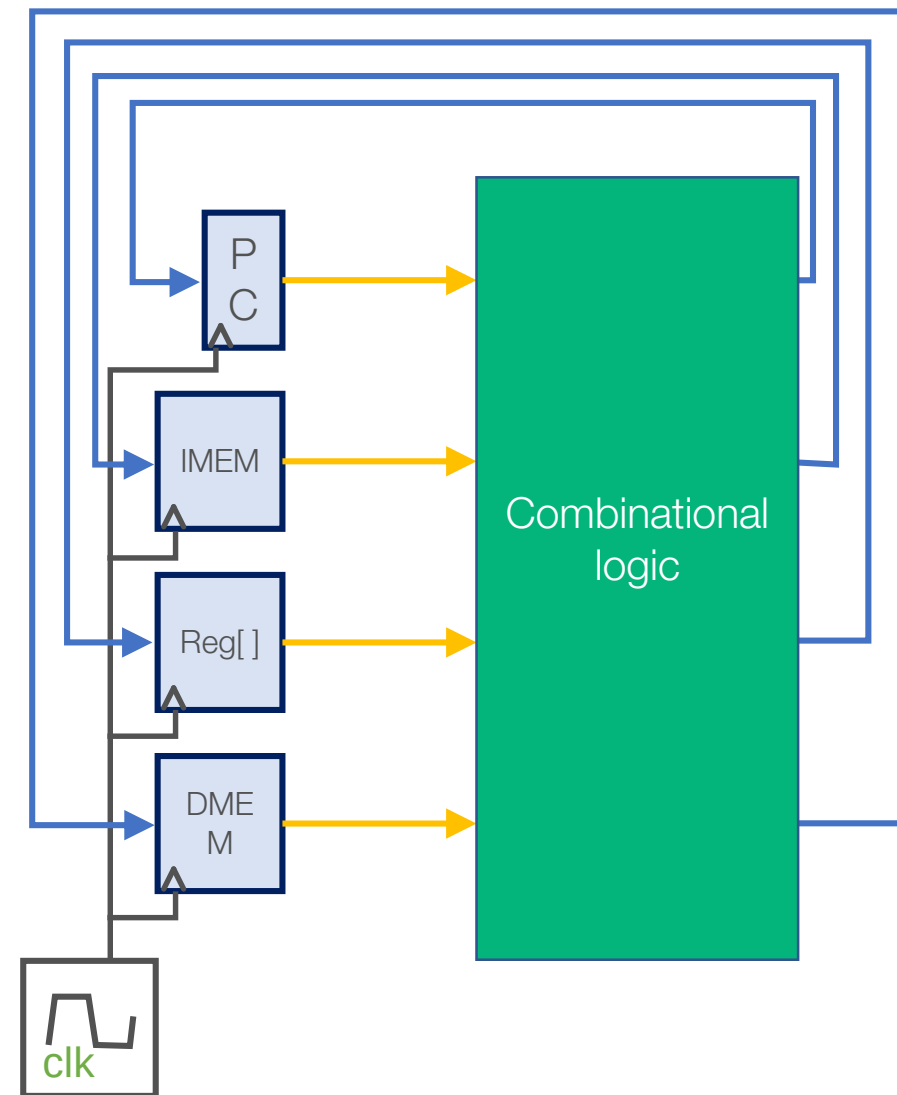
Set of instructions

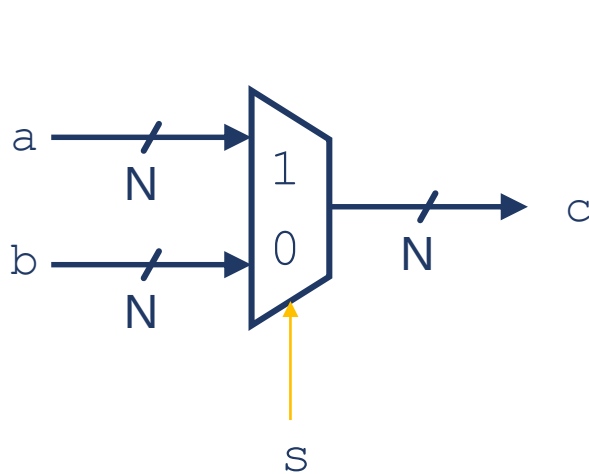


CPU Elements and Stages

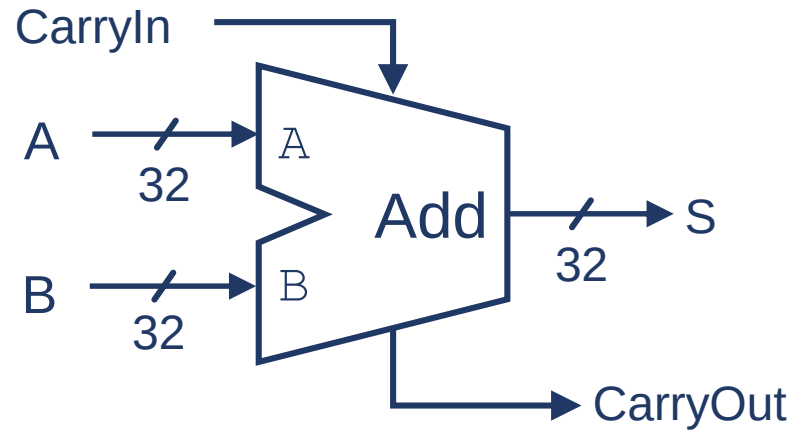
One-Instruction-Per-Cycle RISC V Machine

- The CPU is composed of two types of subcircuits:
 - **Combinational logic blocks** : Adders, Multiplexers (2:1), ALU.
 - **State elements** : PC, RegFile, IMEM, DMEM
- On every tick of the clock, the computer executes one instruction:
 - Current outputs of the *state elements* drive the **inputs to combinational logic**...
 - ...whose outputs settle at the **inputs to the state elements** before the next rising clock edge.
- At the **rising clock edge**:
 - All the **state elements** are updated with the combinational logic outputs...
 - and execution moves to the next clock cycle.

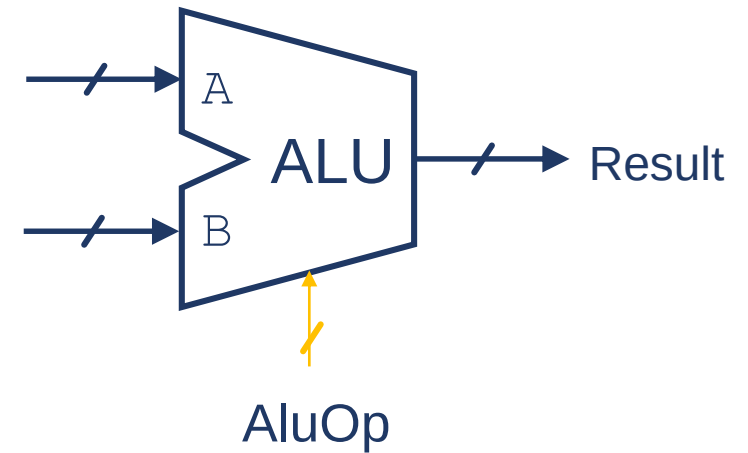




32-bit-wide 2-to-1
MUX



32-Bit
Adder

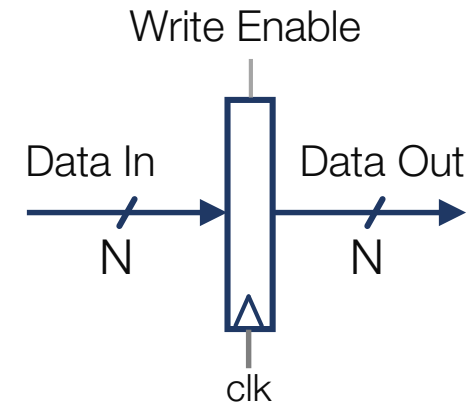
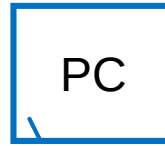


ALU (ALUOp selects from
multiple operations)

State Elements : Program counter

The Program Counter is a
32-bit Register

Program Counter



- **Input:**

- N-bit data input bus
- **Write Enable** “Control” bit (1: asserted, 0: deasserted)

- **Output:**

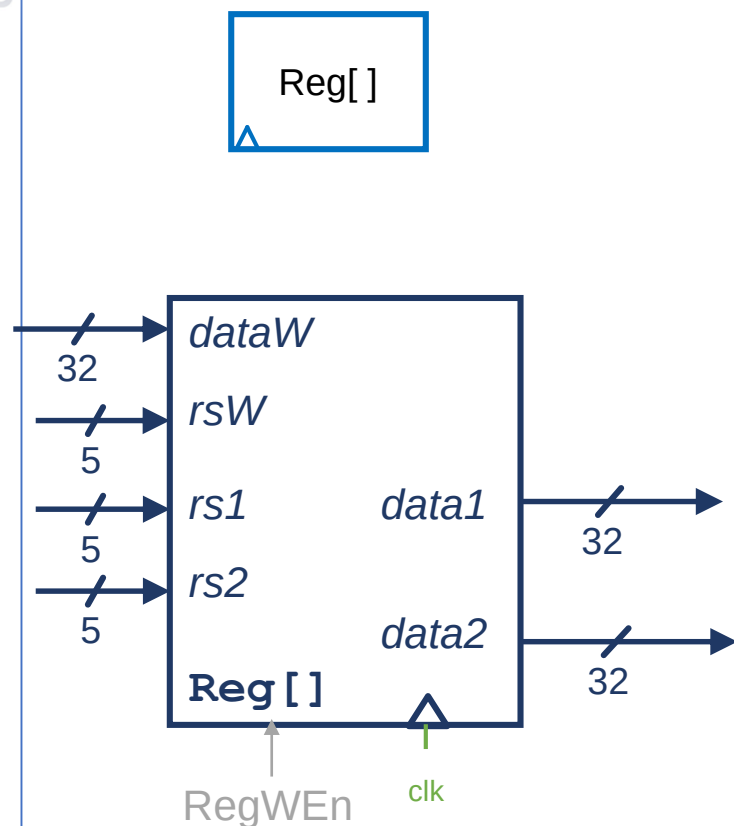
- N-bit data output bus

- **Behavior:**

- If **Write Enable** is 1 on **rising clock edge**, set *Data Out* = *Data In*.
- At all other times, *Data Out* will not change; it will output its current value.

State Elements: Register file

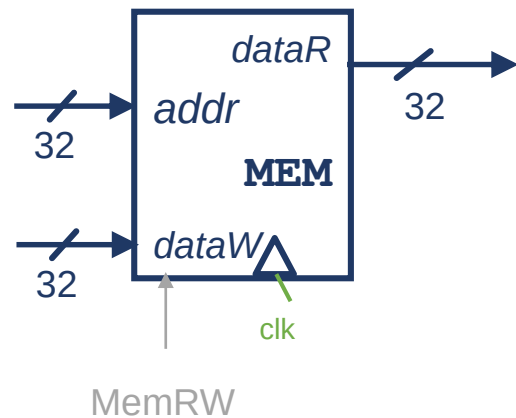
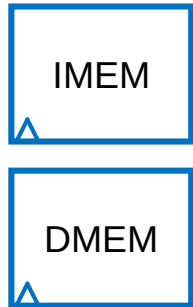
Register File



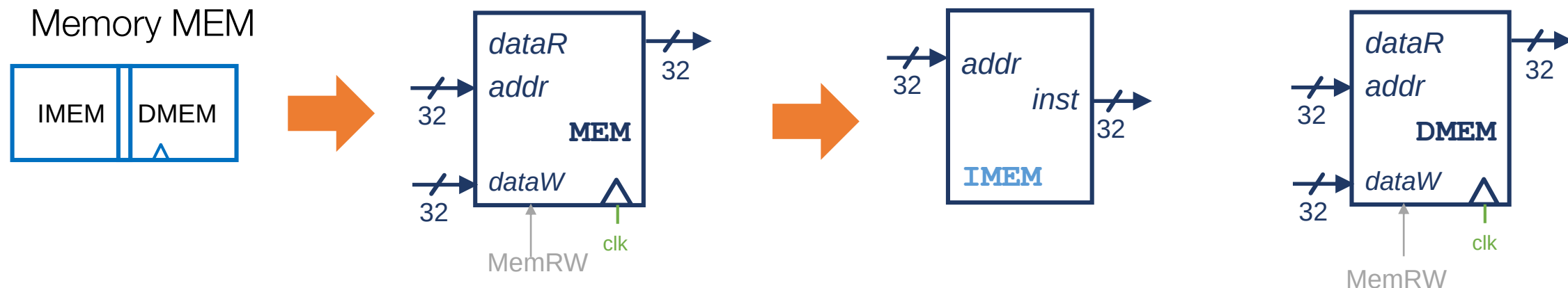
RegFile behaves like a combinational block for read operations!

- Register File (**RegFile**) has 32 registers.
- Input:**
 - One 32-bit input data bus, *dataW*.
 - Three 5-bit select busses, *rs1*, *rs2*, and *rsW* (*rd*).
 - RegWEn* control bit
- Output:**
 - Two 32-bit data output busses
- Registers are accessed via their 5-bit register numbers:
 - R[rs1]*** : *rs1* selects register to put on *data1* bus out.
 - R[rs2]*** : *rs2* selects register to put on *data2* bus out.
 - R[rd]*** : *rsW* selects register to be written via *dataW* when ***RegWEn*=1**.
- Clock behavior:** Write operation occurs on **rising clock edge**.
 - Clock input only a factor on write!
 - All read operations behave like a combinational block:
 - If *rs1*, *rs2* valid, then *data1*, *data2* valid **after** access time.

Memory MEM

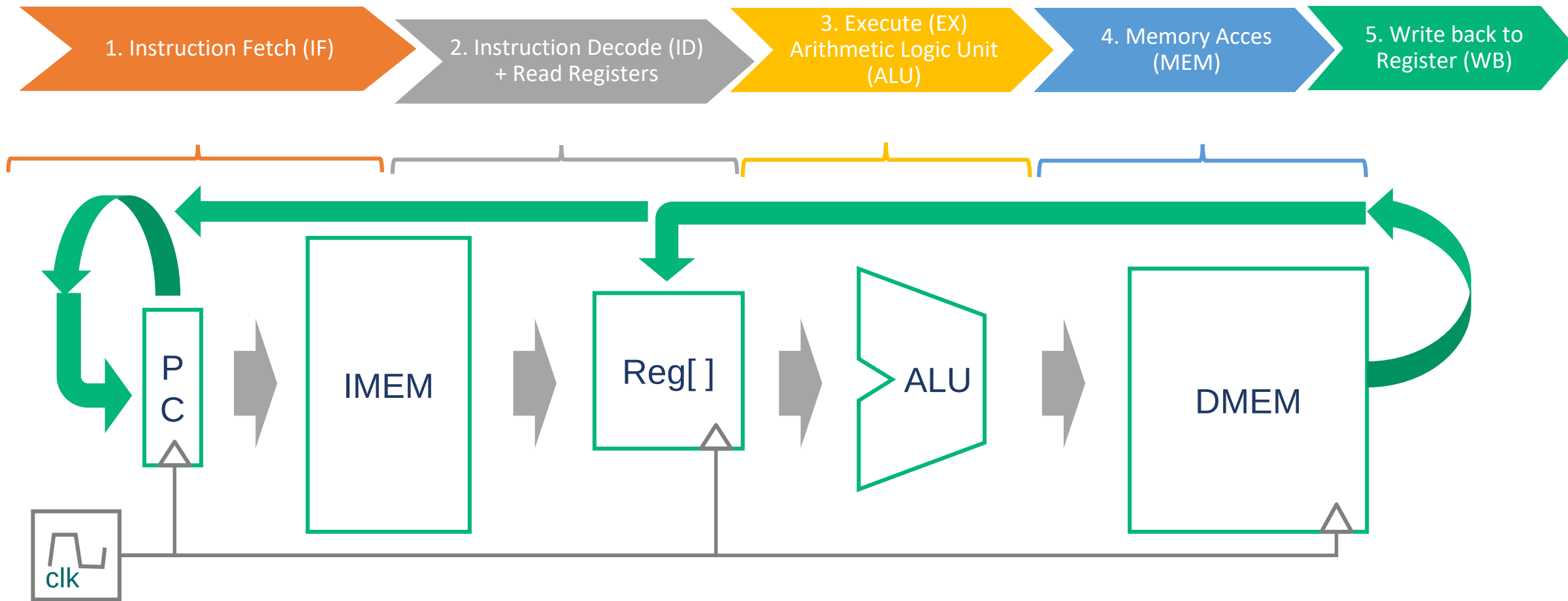


- Memory
 - 32-bit byte-addressed memory space; and
 - Memory access with 32-bit words.
- Memory words are accessed as follows:
 - **Read:** Address *addr* selects word to put on *dataR* bus.
 - **Write:** Set *MemRW*=1. Address *addr* selects word to be written with *dataW* bus.
- Like RegFile, **clock input** is only a factor on write.
 - If *MemRW*=1, write occurs on rising clock edge.
 - If *MemRW*=0 and *addr* valid, then *dataR* valid after access time.
- If *MemRW*=0, **MEM** behaves like a combinational block.



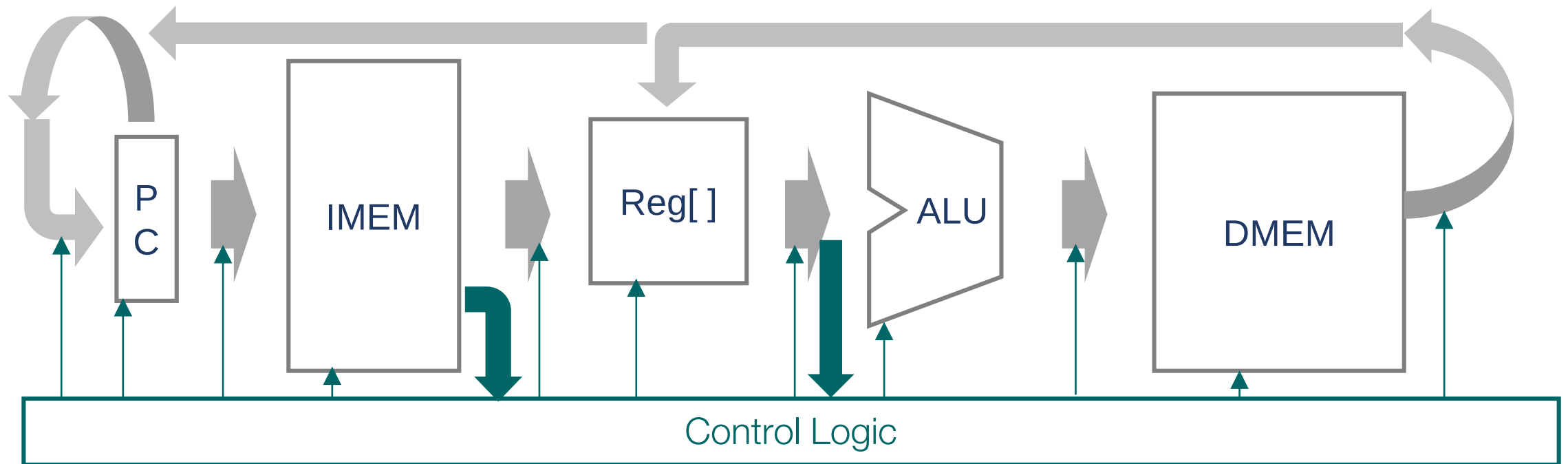
- Memory
 - 32-bit byte-addressed memory space; and Memory access with 32-bit words.
 - Like RegFile, **clock input** is only a factor on write.
 - If **MemRW=1** write occurs on rising clock edge.
 - If **MemRW=0** and **addr** valid, then **dataR** valid after access time.
 - **Current abstraction:** Memory holds both instructions and data in one contiguous 32-bit memory space. In our processor, we'll use **two "separate" memories**:
 - **IMEM:** A read-only memory for fetching instructions.
 - **DMEM:** A memory for loading (read) and storing (write) data words.
 - Because IMEM is read-only, it always behaves like a combinational block:
 - If **addr** valid, then **instr** valid after access time.

5 Basic Stages (Phases) of Instruction Execution



- We will implement a single-cycle processor:
- All stages of one RV32I instruction execute within the same clock cycle.

Not All Instructions Need All 5 Stages!



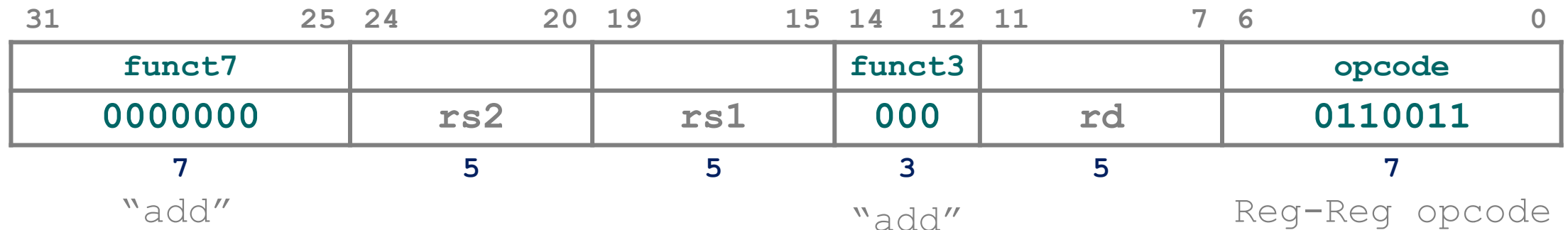
- The control logic selects “needed” datapath lines based on the instruction.
- MUX selector, ALU op selector, write enable, etc.

Arithmetic/Logical Datapath

Implementing the **add** Instruction

- Suppose we had a single instruction in our RISC-V ISA: **add**.

add rd, rs1, rs2



- The add instruction makes **two changes** to processor state:

- RegFile : $\text{Reg}[\text{rd}] = \text{Reg}[\text{rs1}] + \text{Reg}[\text{rs2}]$
- PC : $\text{PC} = \text{PC} + 4$

31	25	24	20	19	15	14	12	11	7	6	0				
funct7					rs2			rs1		funct3		rd		0110011	

$$PC = PC + 4$$

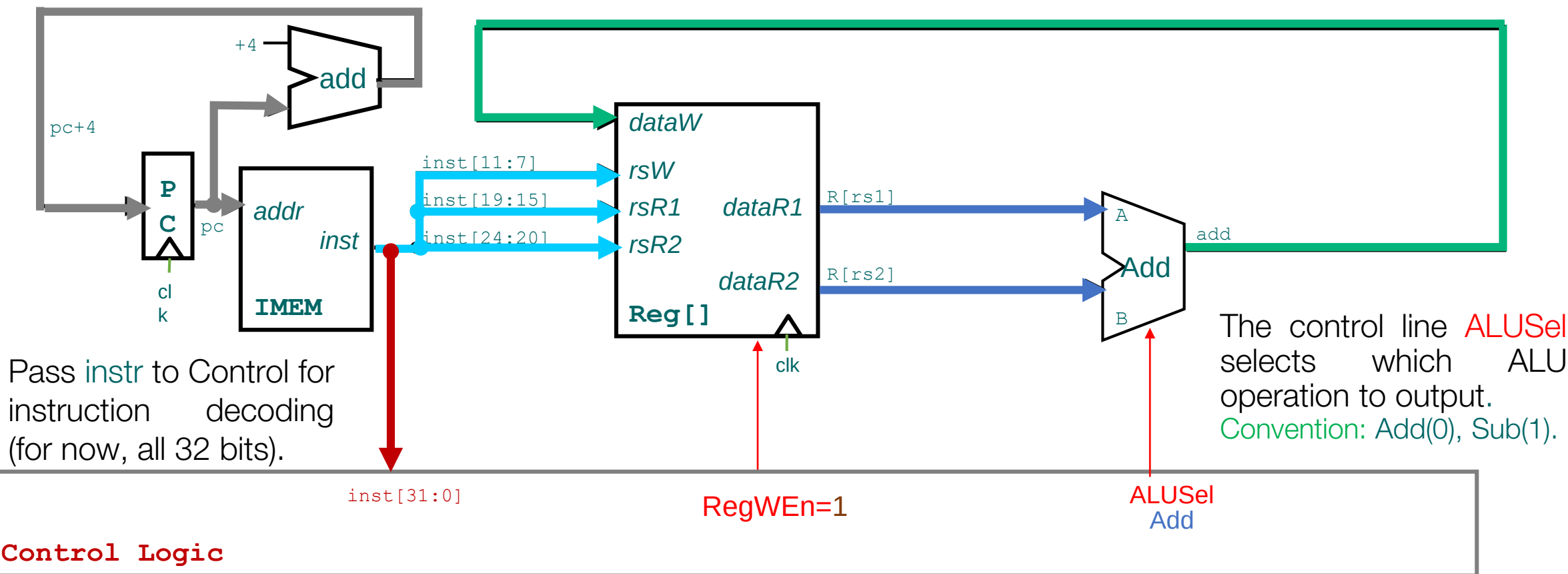
$$R[rd] = R[rs1] + R[rs2]$$

Increment PC to next instruction.

Split instruction to index into RegFile.

Feed read register values into Add.

Write Add output to destination register.



Implementing the **sub** Instruction

- What if our ISA now had two instructions: add/sub?

funct7		funct3		opcode		
0000000	rs2	rs1	000	rd	0110011	add
0100000	rs2	rs1	000	rd	0110011	sub

sub rd, rs1, rs2

- sub is almost the same as add, except now the ALU subtracts operands instead of adding them:
 - RegFile : $\text{Reg}[\text{rd}] = \text{Reg}[\text{rs1}] - \text{Reg}[\text{rs2}]$
 - PC : $\text{PC} = \text{PC} + 4$
- Instruction bit inst[30] selects between add/sub.

31	25	24	20	19	15	14	12	11	7	6	0
0100000	rs2	rs1	funct3	rd	0110011						

$$PC = PC + 4$$

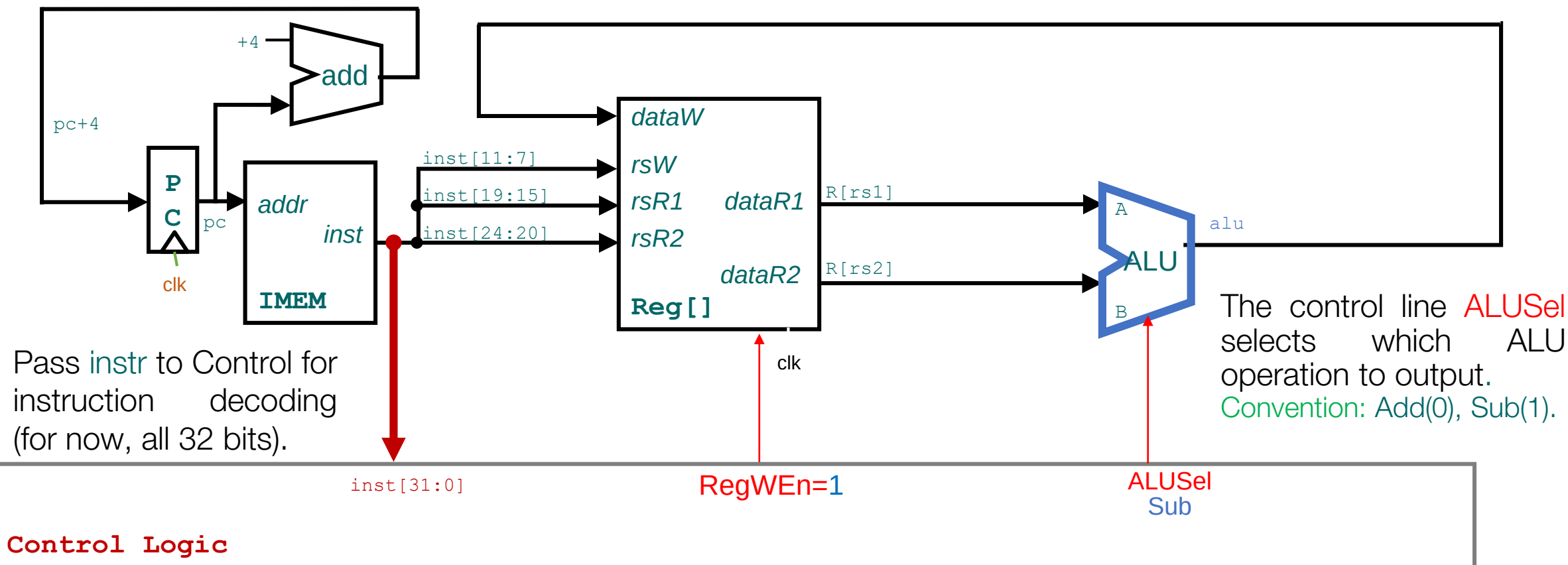
$$R[rd] = R[rs1] - R[rs2]$$

Increment PC to next instruction.

Split instruction to index into RegFile.

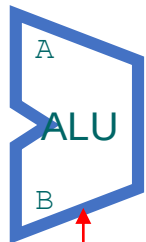
Feed read register values *into ALU*.

Write *ALU output* to destination register.



Supporting all R-Type Instructions

funct7		funct3		opcode		
0000000	rs2	rs1	000	rd	0110011	add
0100000	rs2	rs1	000	rd	0110011	sub
0000000	rs2	rs1	001	rd	0110011	sll
0000000	rs2	rs1	010	rd	0110011	slt
0000000	rs2	rs1	011	rd	0110011	sltu
0000000	rs2	rs1	100	rd	0110011	xor
0000000	rs2	rs1	101	rd	0110011	srl
0100000	rs2	rs1	101	rd	0110011	sra
0000000	rs2	rs1	110	rd	0110011	or
0000000	rs2	rs1	111	rd	0110011	and



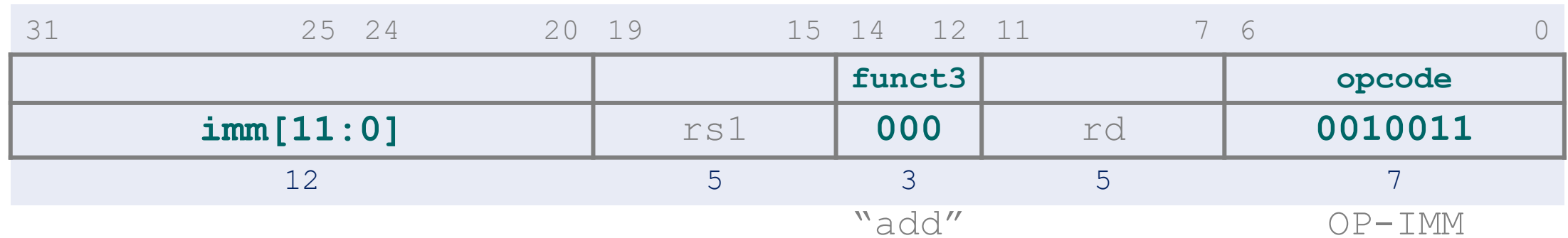
ALUSel
add,sub,xor,and,or,
slt,sltu,sll,sra,srl

The **Control Logic** decodes **funct3**, **funct7** instruction fields and selects appropriate **ALU** function by setting the control line **ALUSel**.

Implementing the **addi** Instruction

- Suppose we add a new instruction: **addi**, RV32I I-Format:

addi **rd, rs1, imm**



- The **addi** instruction updates the same two states as before.
But we now need to build an **immediate imm**!

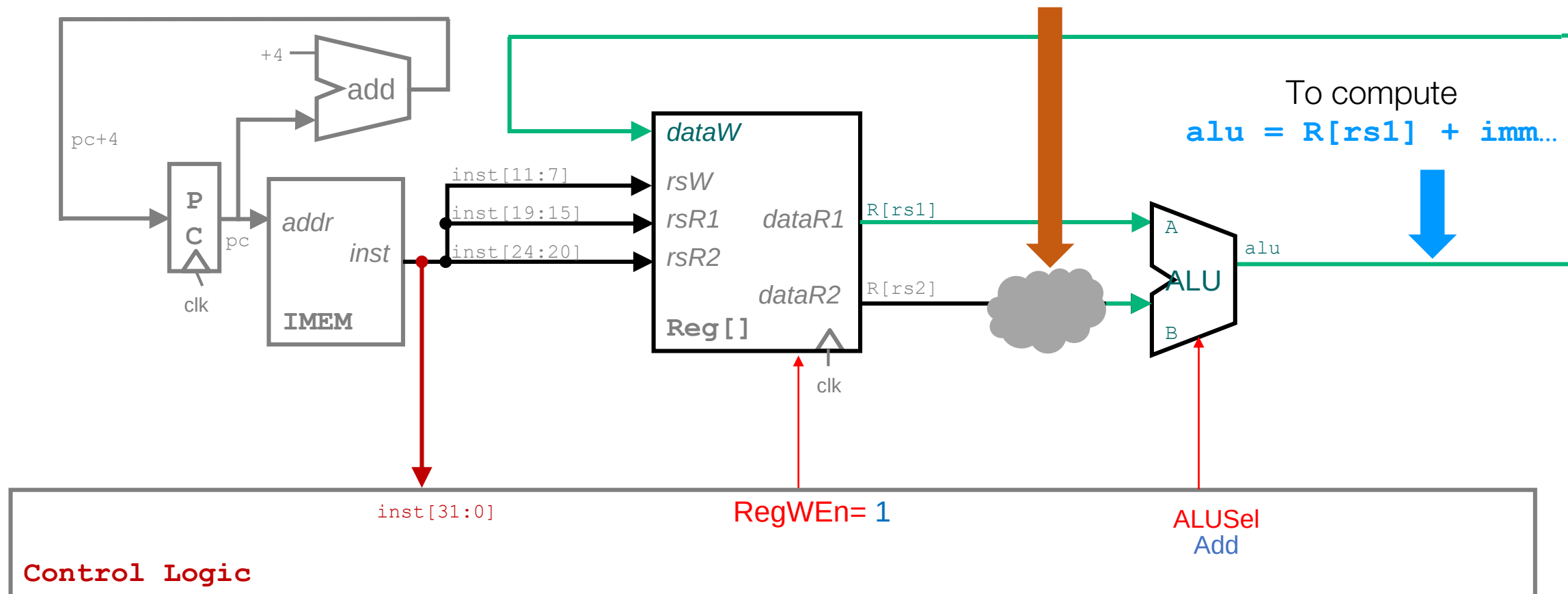
- RegFile : $\text{Reg}[\text{rd}] = \text{Reg}[\text{rs1}] + \text{imm}$
- PC : $\text{PC} = \text{PC} + 4$

What Does the Datapath Need for addi?

$$PC = PC + 4$$

$$R[rd] = R[rs1] + \text{imm}$$

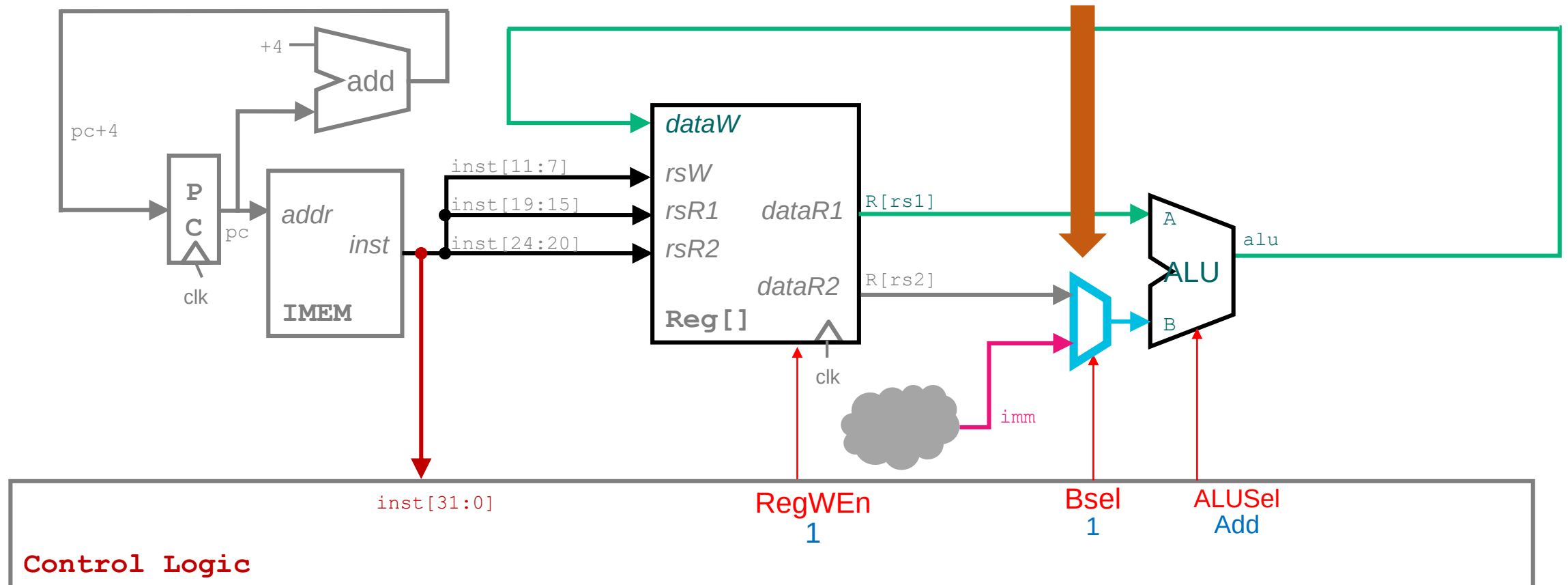
...we should feed in an **immediate**
ALU input B instead of $R[rs2]$.



ensia New MUX to Select Immediate for ALU

GUZIO

1. Control line **Bsel=1** selects the generated immediate **imm** for ALU input B.

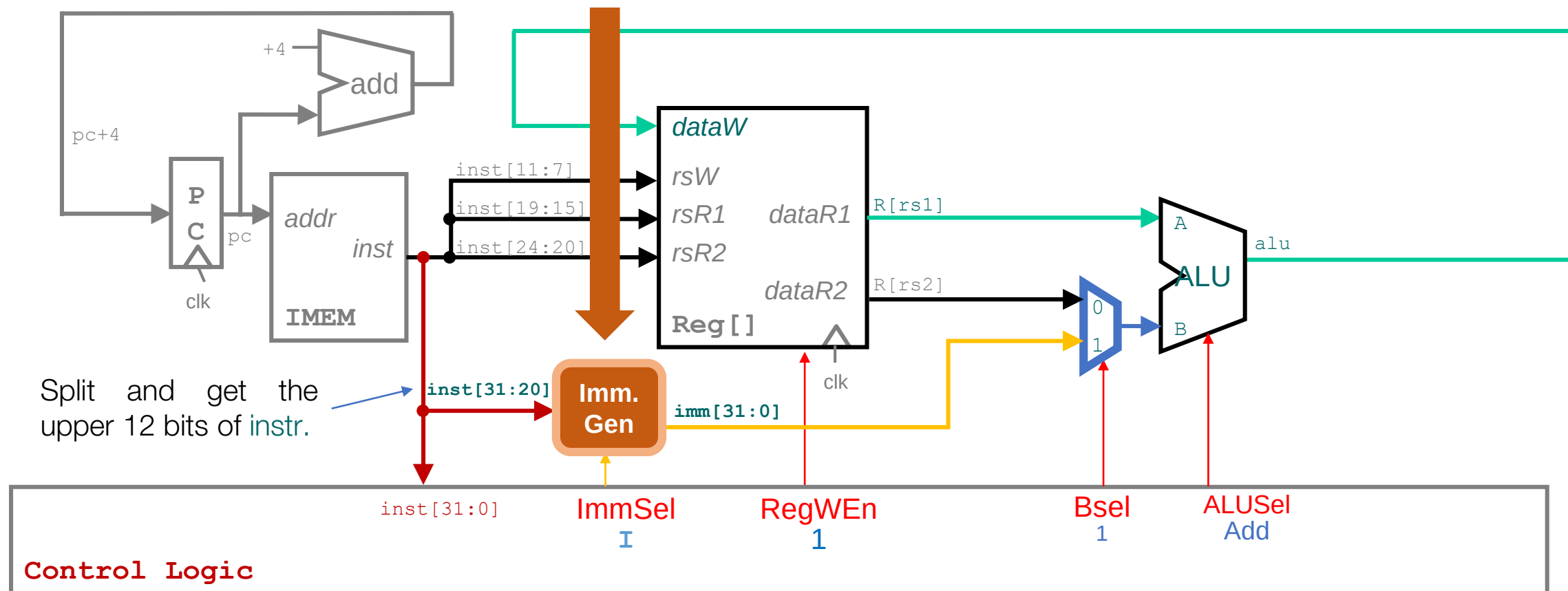


ensia New Block to Generate 32-bit Immediate

GUZIS

2. Immediate Generation Block builds a 32-bit immediate *imm* from instruction bits.

1. Control line *Bsel=1* selects the generated immediate *imm* for ALU input B.



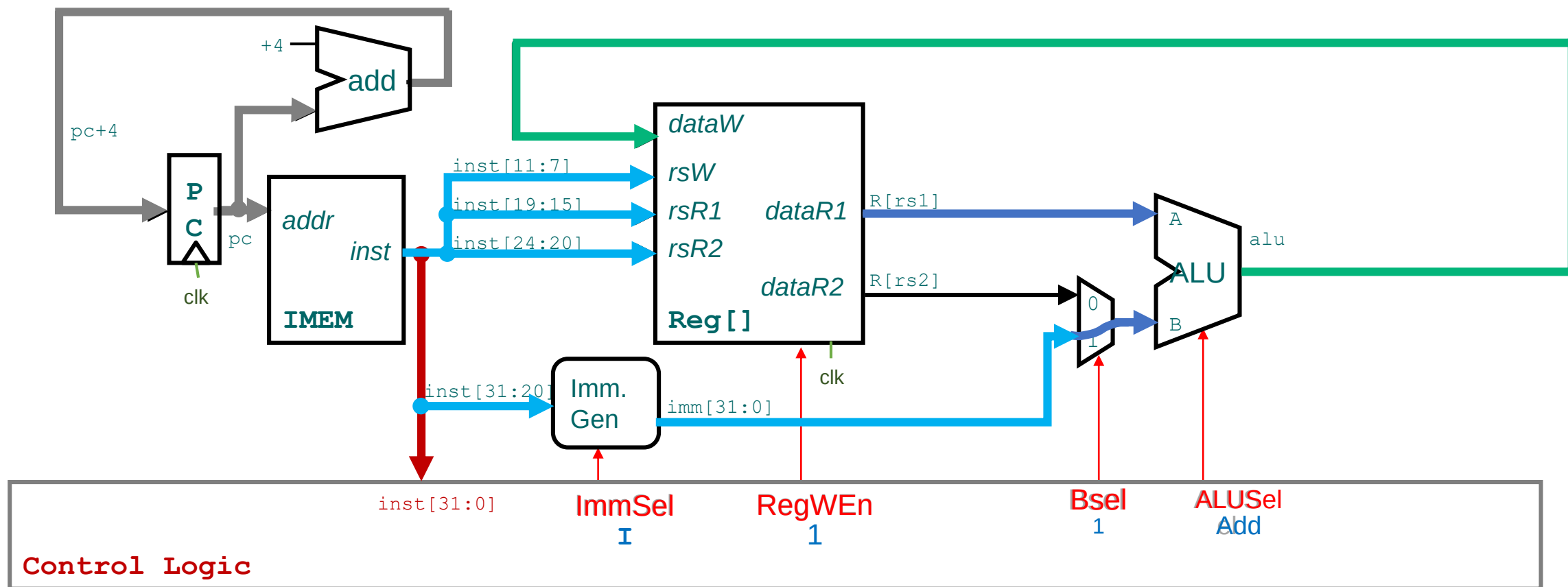
ensia Datapath for addi

Increment PC to next instruction.

Immediate Generation Block builds a 32-bit immediate *imm* from instruction bits.

Control line *Bsel=1* selects the generated immediate *imm* for ALU input B.

Write ALU output to destination register.

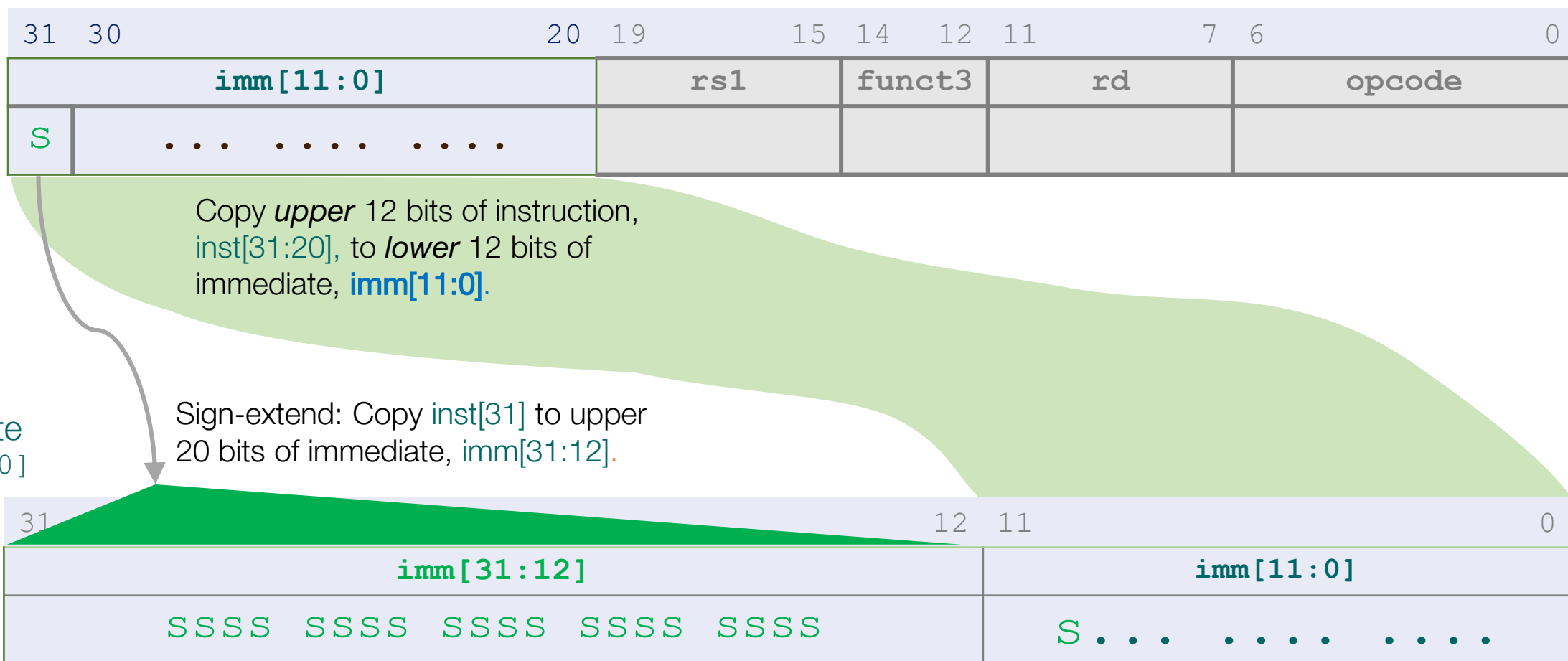
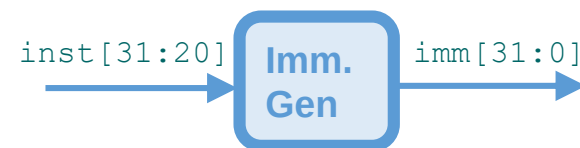


- Data *inst[24:20]* still feeds into *Reg[]*, which still outputs *R[rs2]*. However, control *Bsel=1* means *R[rs2]* data line is ignored.

ensia Immediate Generation Block Design

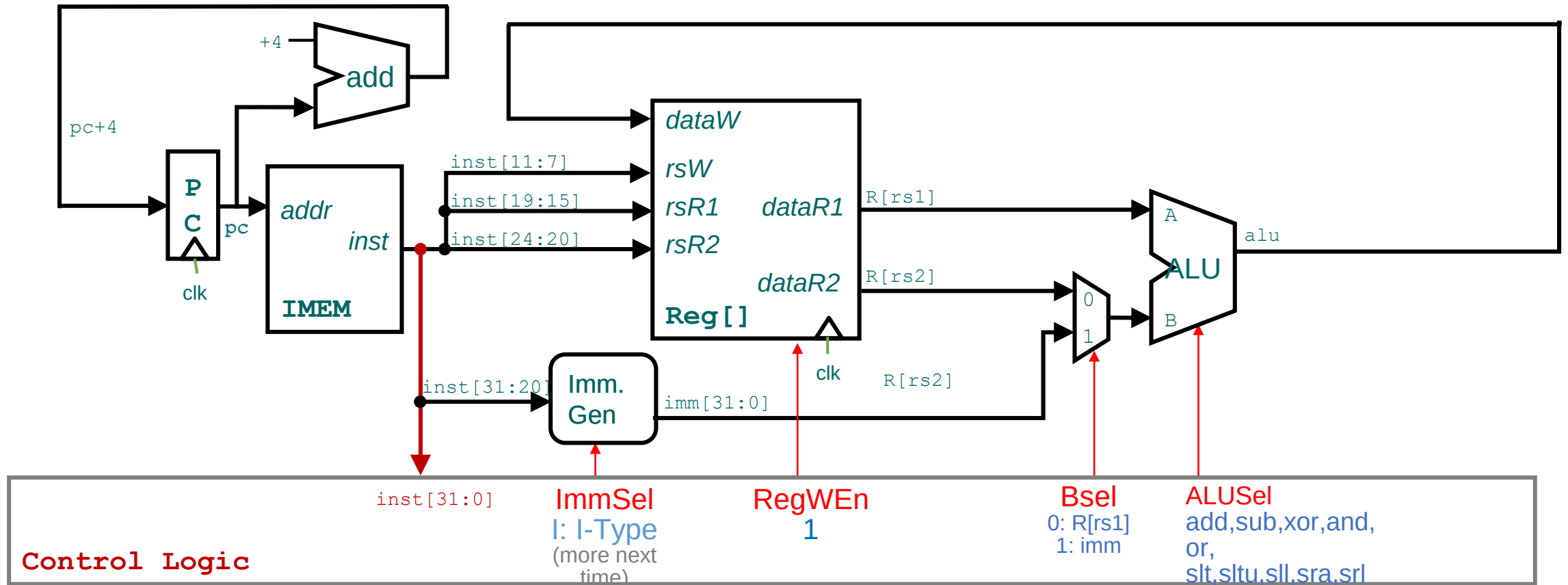
GUZIS

Instruction
 $inst[31:0]$



Summary: Arithmetic/Logical Datapath

- All data lines carry information.
- Control logic determines what is “useful/needed” vs. what is “ignored.”
 - e.g., ALUSel: chooses ALU operation; Bsel: chooses register/immediate for ALU input B.



Load/Store Datapath

Implementing the **lw** Instruction

- **lw** uses I-Format: `lw x14, 8(x2)`

31	20	19	15	14	12	11	7	6	0	
imm[11:0]				rs1		funct3	rd		opcode	
000000001000				00010		010	01110		0000011	
12				5		3	5		7	

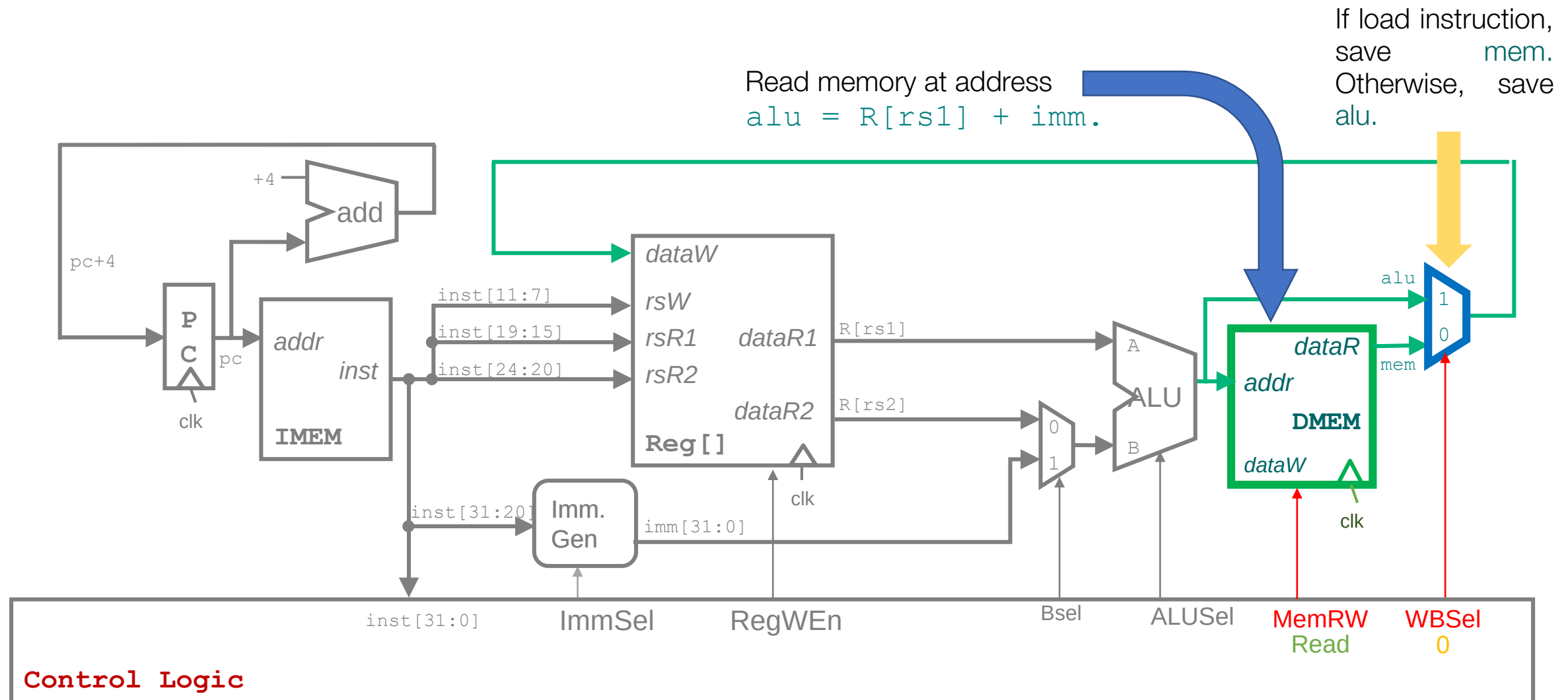
"load word"

LOAD

- Similar datapath to **addi**, but creates an address (not the final value stored).
 - **addr** = (Base register **rs1**) + (sign-extended **imm** offset)
- State element access now includes a **memory read**!
 - **DMEM** : read word at address **addr**
 - **RegFile** : Reg[rs1] # read; Reg[rd] # write
 - **PC** : PC = PC + 4

Saving Memory Read to RegFile

GUZ19



ensia LOAD Datapath

GUZIS

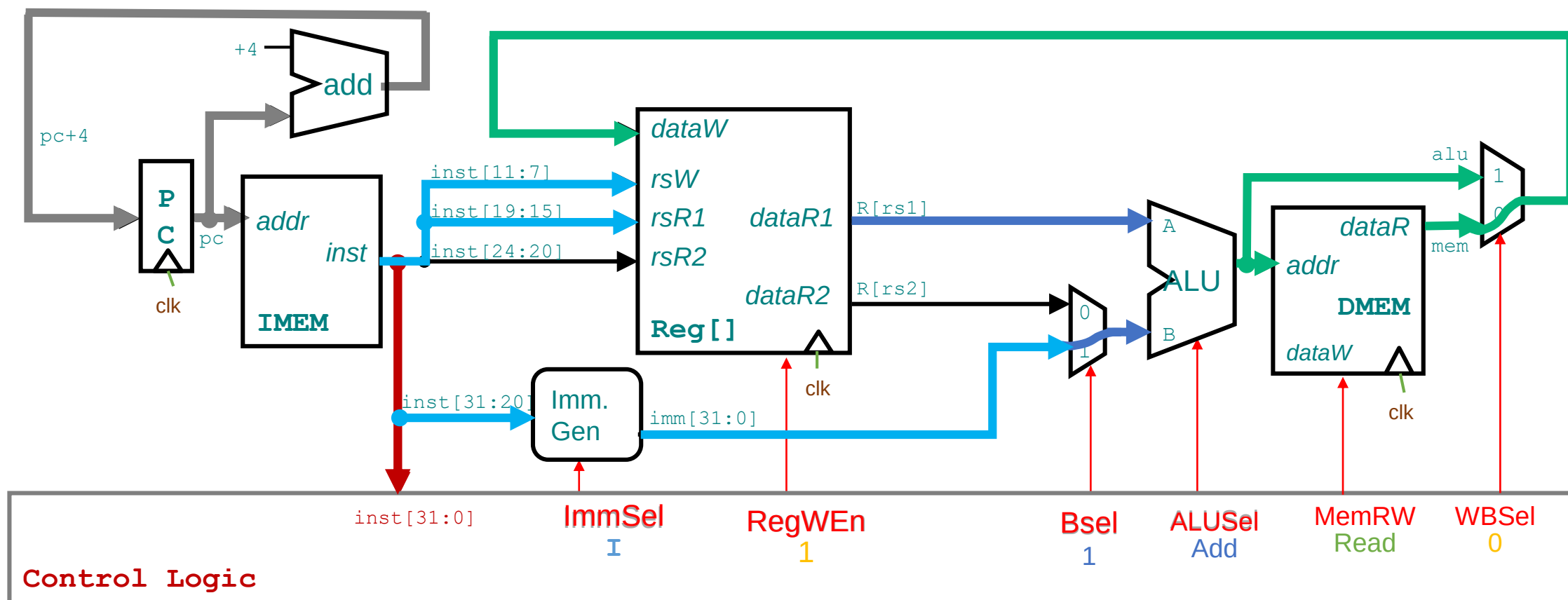
Increment PC to
next instruction.

Immediate Generation
Block builds a 32-bit
immediate *imm*.

ALU computes
address
 $alu = R[rs1] + imm$.

Read memory at
address *alu*.

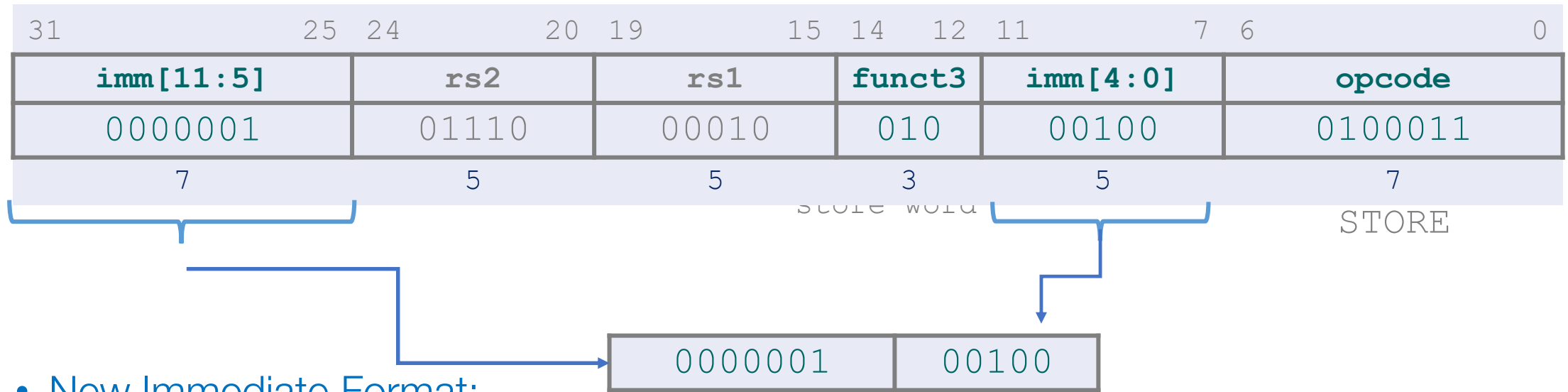
Write loaded memory
value *mem* to register.



Load uses all five stages of the datapath!

Implementing the **sw** Instruction

- S-Format: `sw x14, 36(x2)`



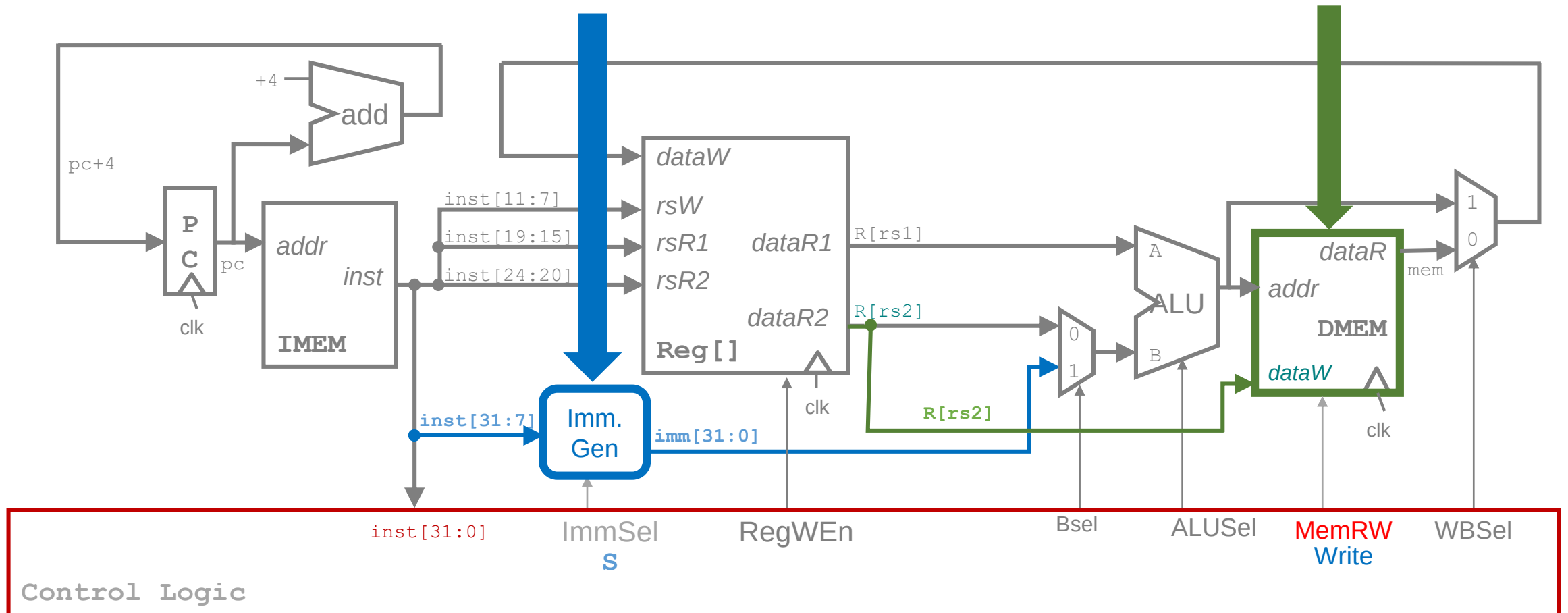
- New Immediate Format:
 - $\text{addr} = (\text{Base register } rs1) + (\text{sign-extended imm offset})$
- State Elements Accessed:
 - DMEM : (write $R[rs2]$ to word at address addr)
 - RegFile : $R[rs1]$ (base address), $R[rs2]$ (value to store) } No RegFile write!
 - PC : $PC = PC + 4$

ensia Updated Blocks for sw

GUZ19

Control **ImmSel** selects how to generate immediate type: **I**, **S**.

Control **MemRW=Write** saves **rs2** to memory on the next rising clock edge.



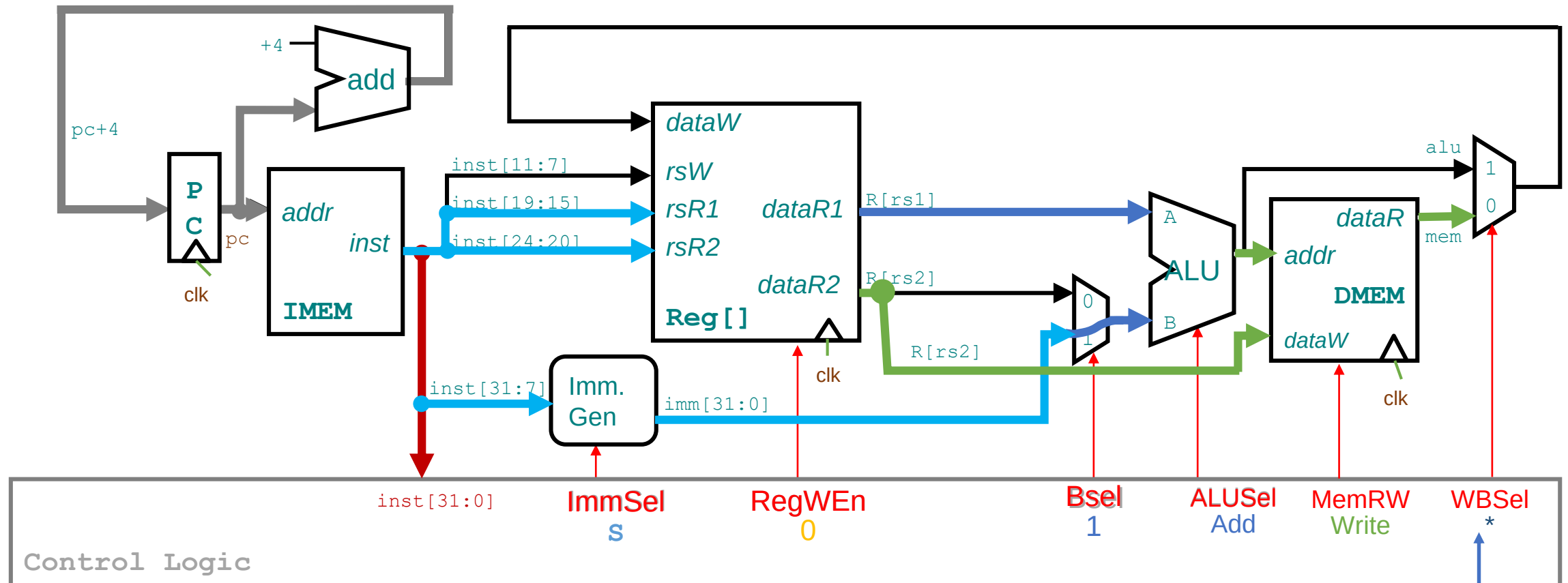
ensia STORE Datapath

Increment PC to
next instruction.

Build *imm* from
S-type instruction.

ALU computes address
 $alu = R[rs1] + imm$.

Write memory at
address *alu*.



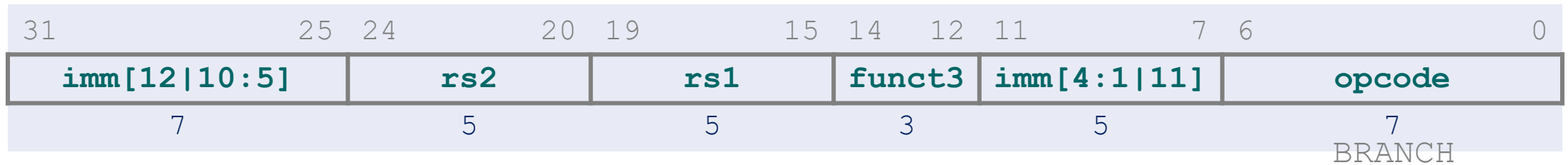
RegWEn=0 means no write back to RegFile, so “*don’t care*” (*) about WBSEL’s value.

B-Type (Branches) Datapath

B-Format: Conditional Branch

- B-Format close to S-Format:

opname rs1, rs2, Label



- New Immediate Format!
- PC state element now conditionally changes:
 - RegFile : R[rs1], R[rs2] (read only, for branch comparison)
 - PC : $PC = PC + imm$ (if branch taken)
 $PC = PC + 4$ (otherwise, not taken)

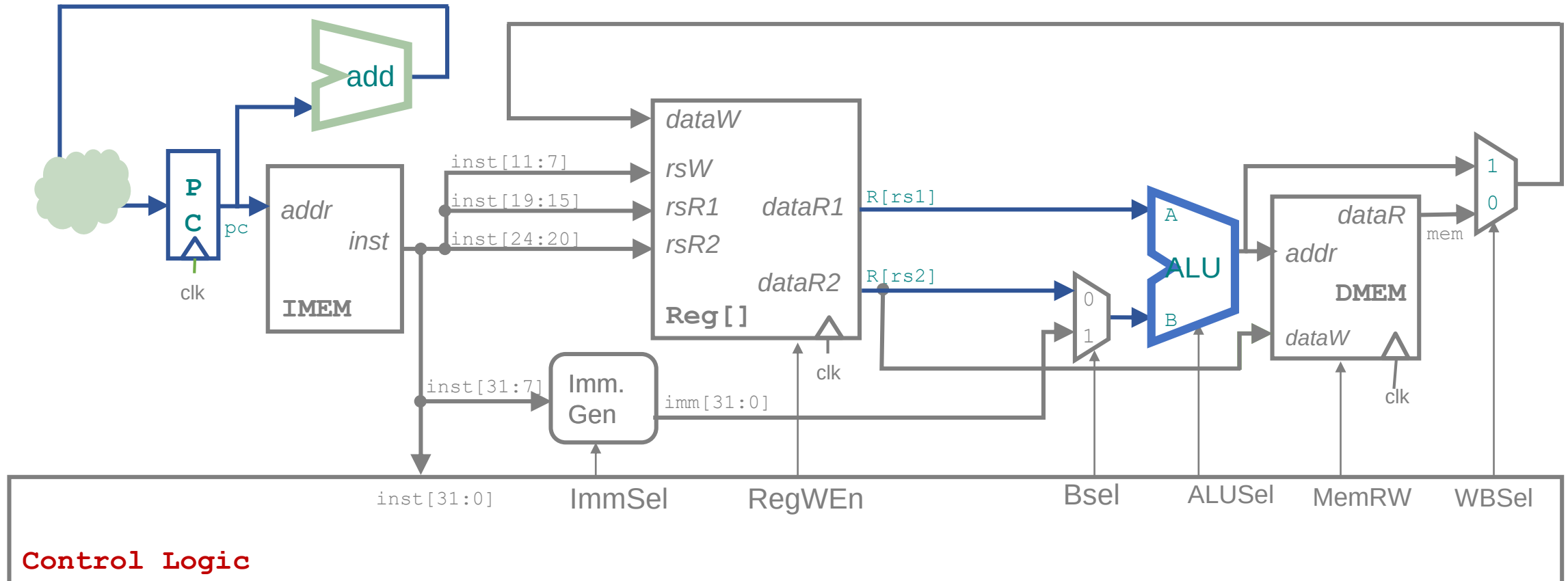
What do Branches Need to Compute?

PC = PC + imm ?

PC = PC + 4 ✓

Compare R[rs1] , R[rs2] ?

Only one ALU accessible in the same clock cycle. Need more hardware!!

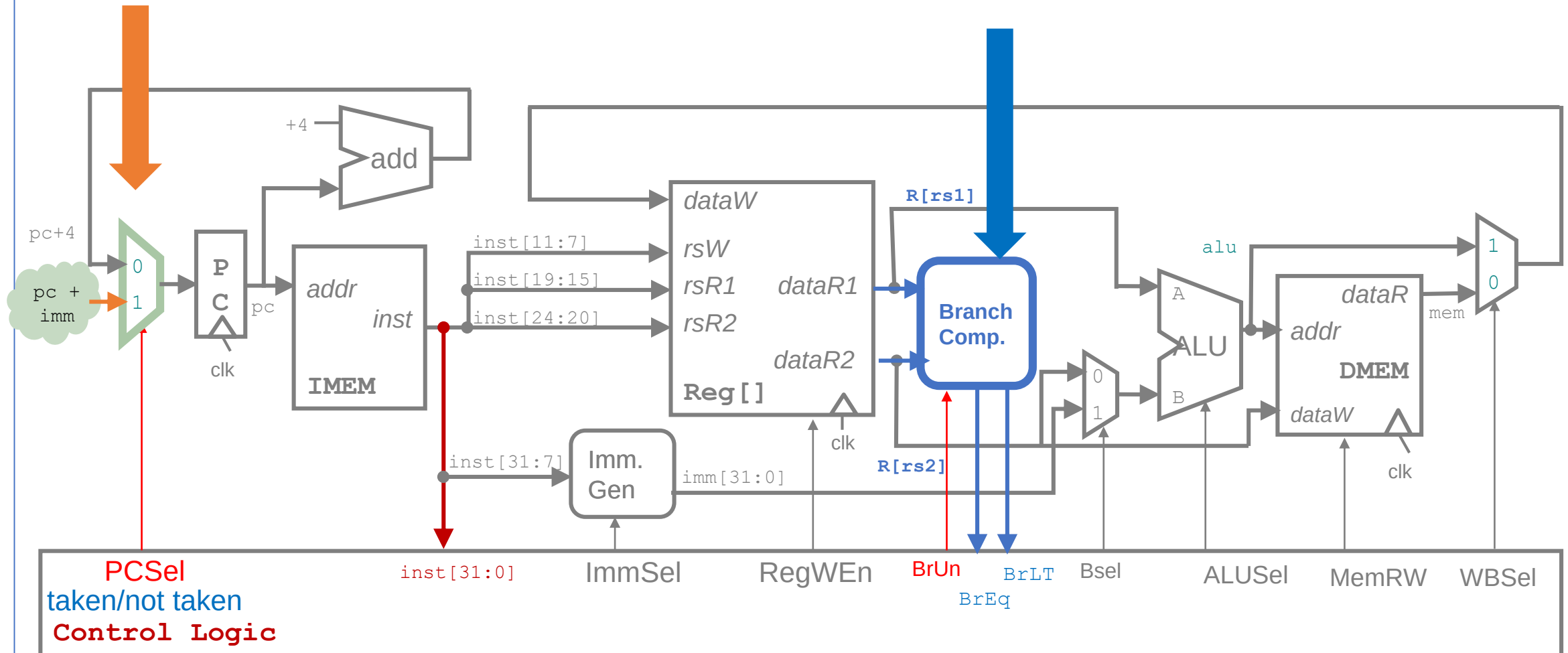


ensia The Branch Comparator Block

GUZIO

Update PC based
on branch result.

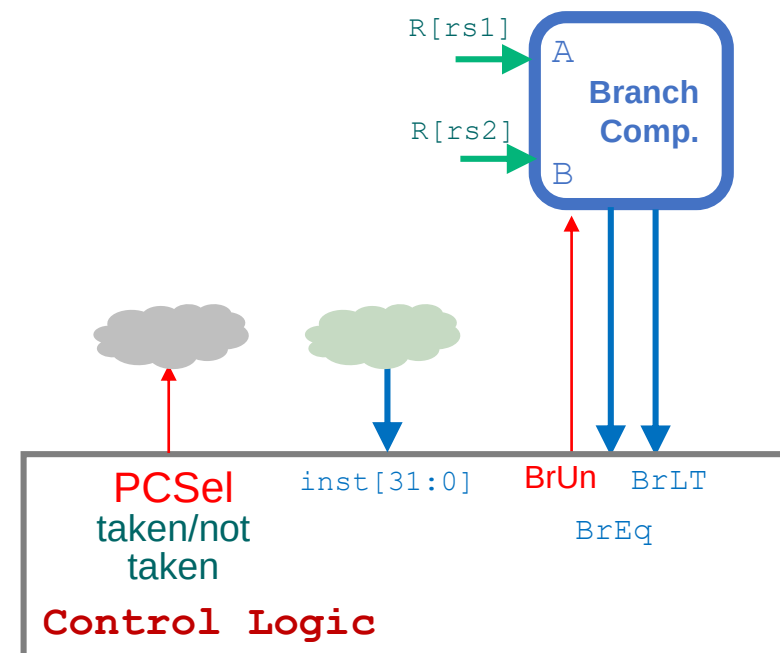
Compare $R[rs1], R[rs2]$,
feed result to Control Logic.



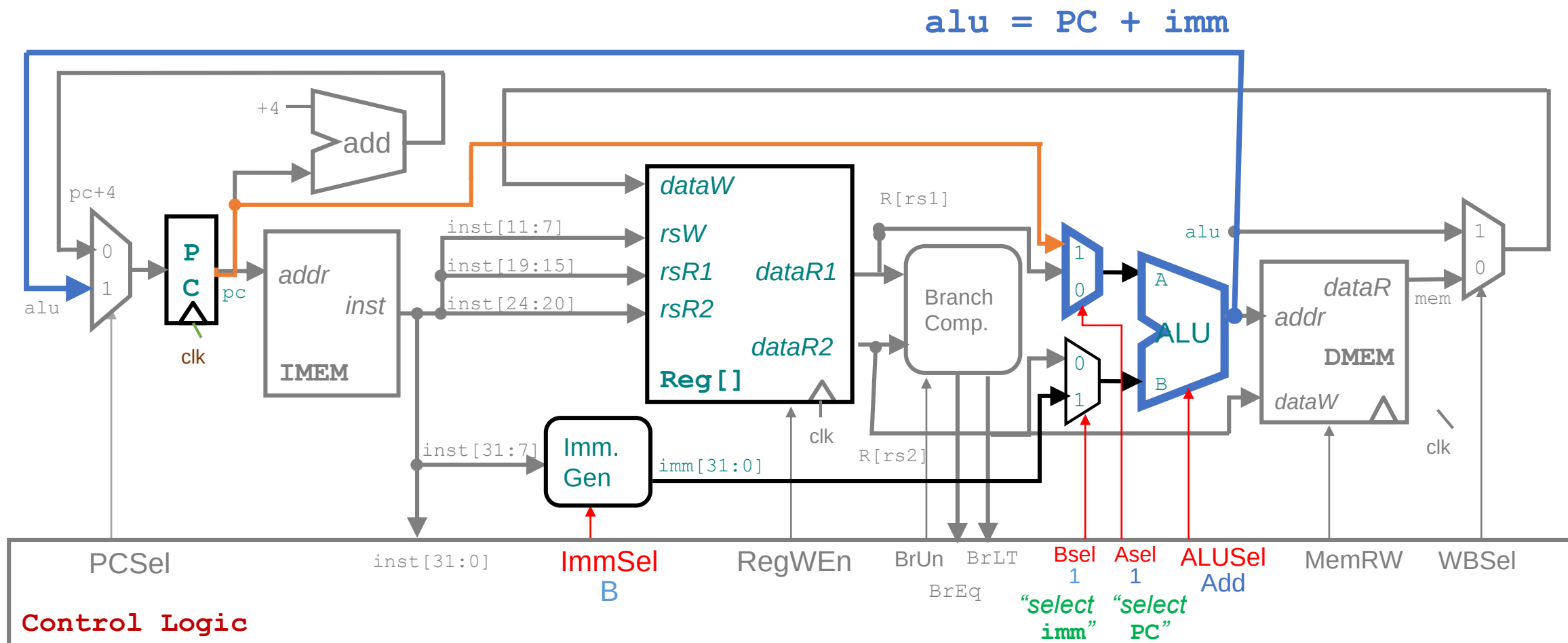
BrUn: Control; BrLT, BrEq: Data

The Branch Comparator Block

- The Branch Comparator is a combinational logic block.
 - **Input:**
 - Two data busses **A** and **B** (datapath **R[rs1]** and **R[rs2]**, respectively)
 - **BrUn** ("Branch Unsigned") control bit
 - **Output:**
 - **BrEq** flag: 1 if $A == B$
 - **BrLT** flag: 1 if $A < B$.
 - Unsigned comparison if **BrUn**=1, signed otherwise.
 - **Control Logic:**
 - Set **BrUn** based on current instruction, **inst[31:0]**.
 - Set **PCSel** based on branch flags **BrLT**, **BrEq**.
 - **Examples**
 - **blt**:
 - If **BrLT**=1 and **BrEq**=0, then **PCSel**=taken.
 - **bge**: $(A \geq B) = \overline{A < B}$
 - If **BrLT**=0, then **PCSel**=taken.



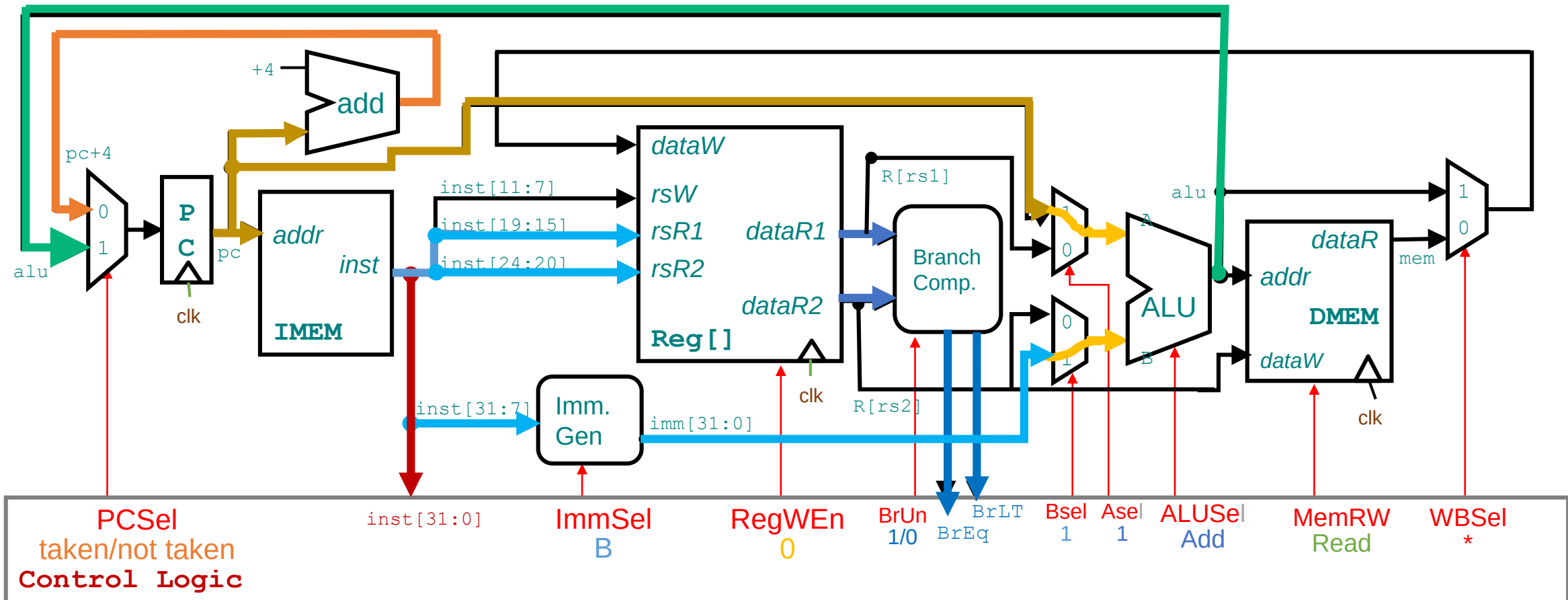
- To compute the address to branch to, use the ALU:



Branch Datapath

If $PCSel=taken$, update PC to ALU output. Else, update to next instruction $PC + 4$.

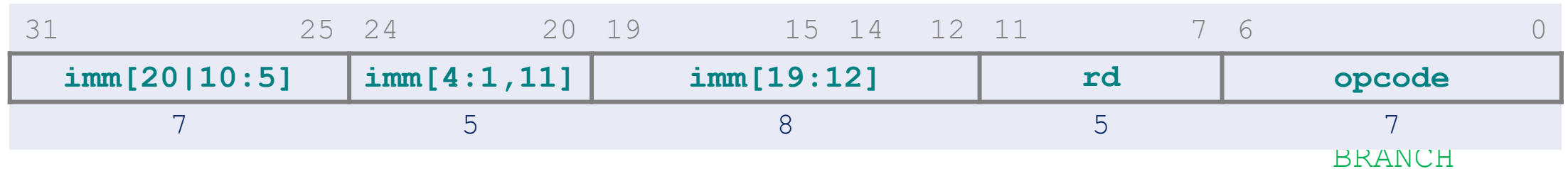
- Build imm from B-type instruction.
- Compute branch; feed to Control.
 - Compute $PC + imm$.
 - Don't write to registers.
 - Don't write to memory.



J-Format: **Unconditional Jump**

- J-Format:

jal rd, Label



- State Elements updated:

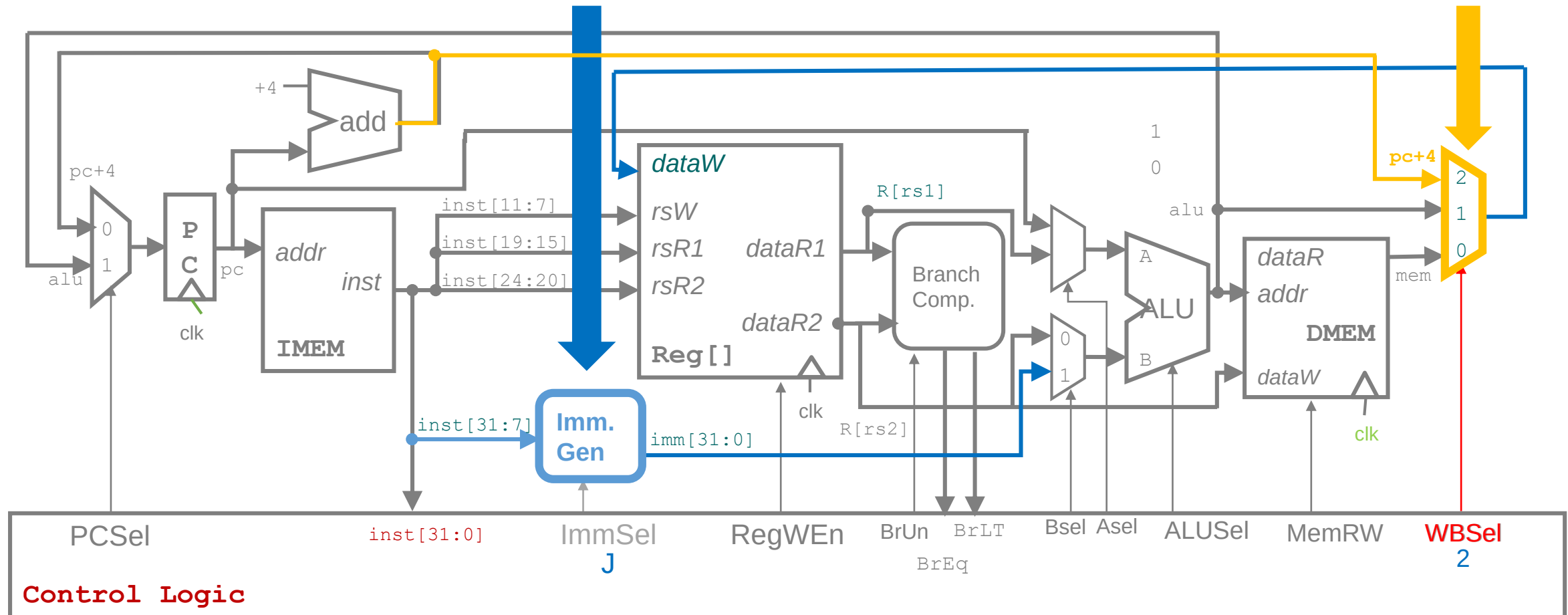
- PC $\mathbf{PC = PC + imm}$ (unconditional PC-relative jump)
- RegFile $\mathbf{rd = PC + 4}$ } Save return address to RegFile destination register.

Block Updates Needed for JAL

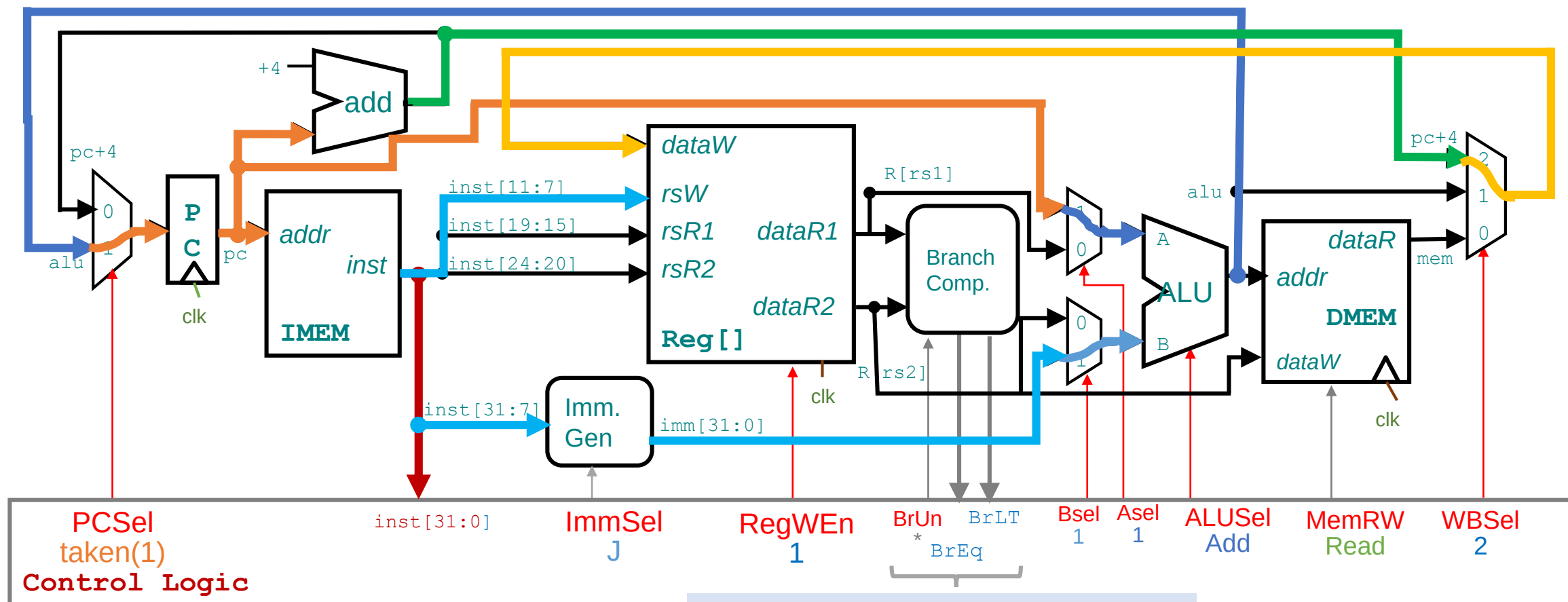
$$\begin{aligned} \text{PC} &= \text{PC} + \text{imm} \\ \text{rd} &= \text{PC} + 4 \end{aligned}$$

Immediate Generation Block needs to support J-Types: 20-bit

To save $\text{rd} = \text{PC} + 4$, **WBSEL** now controls a 3-input MUX.



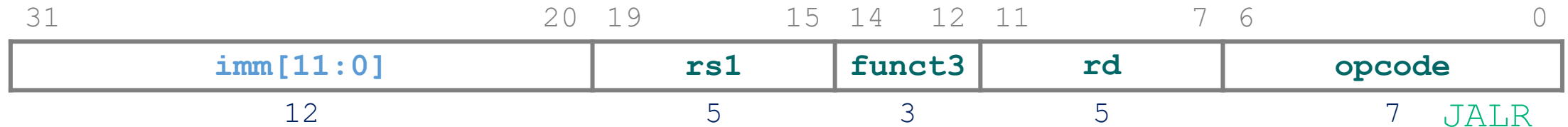
- Feed PC into blocks.
 - Write ALU output to PC.
- Generate byte offset **imm** for 20-bit PC-relative jump.
- Compute $PC + imm$.
Don't write to memory.
- Write $PC + 4$ to destination register.



I-Format Instruction Layout: **jlr**

- **jlr** uses I-Format:

`jlr rd,rs1,imm`



- Two changes to state:
 - **PC** : $PC = R[rs1] + imm$ (absolute addressing)
 - **RegFile** : $R[rd] = PC + 4$
- I-Format means **jlr** uses the same immediates as arithmetic/loads!
 - In other words, **imm** is already a byte offset.

Control ImmSel is based on instruction format, not instruction. So far: I,S,B,J

JALR Datapath

$$PC = R[rs1] + imm$$

$$R[rd] = PC + 4$$

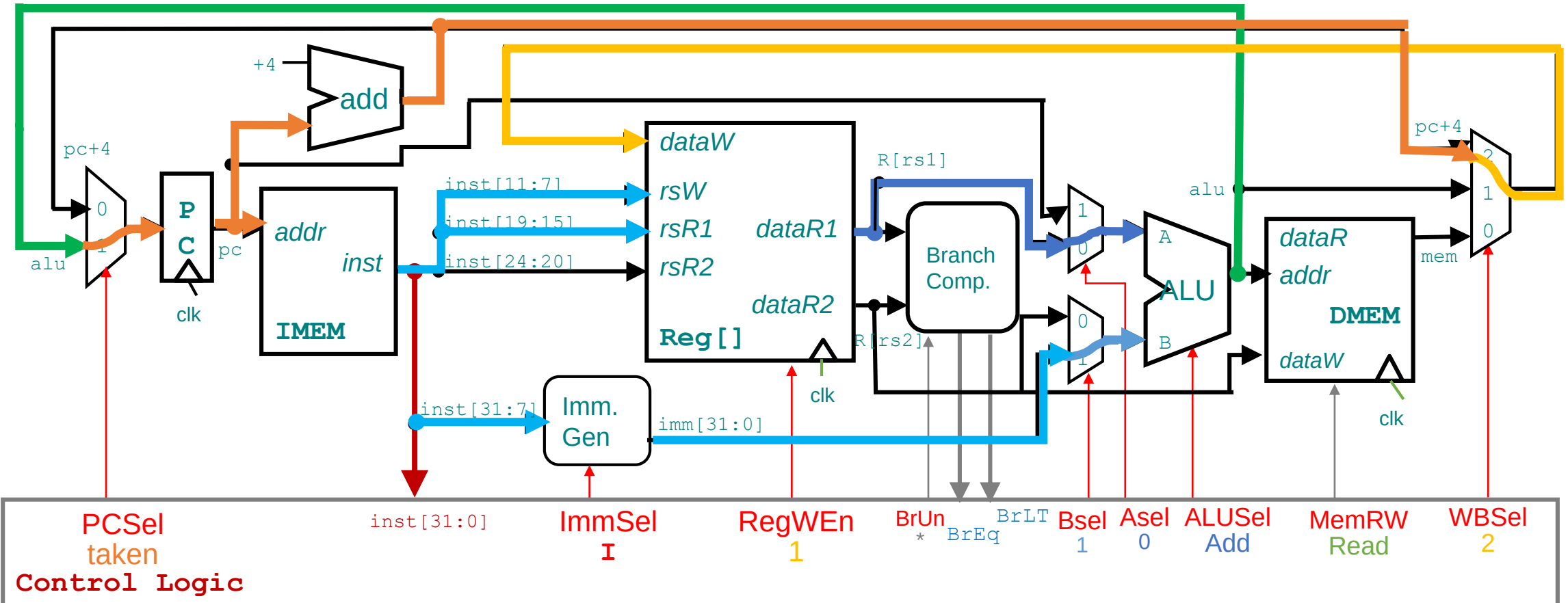
- Feed PC into blocks.
- Write ALU output to PC.

Generate 12-bit imm
(I-Format).

Compute $rs1 + imm$.

Don't write to
memory.

Write $PC + 4$ to
destination register.

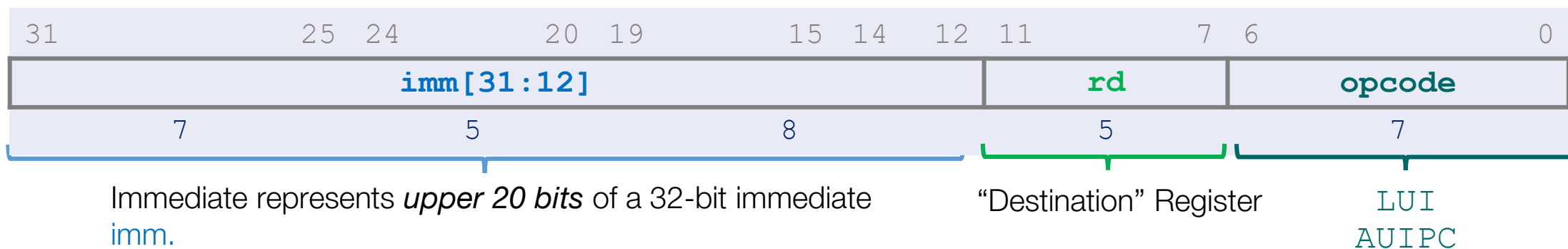


U-Types

U-Format Instruction Layout

- Upper Immediate instructions:

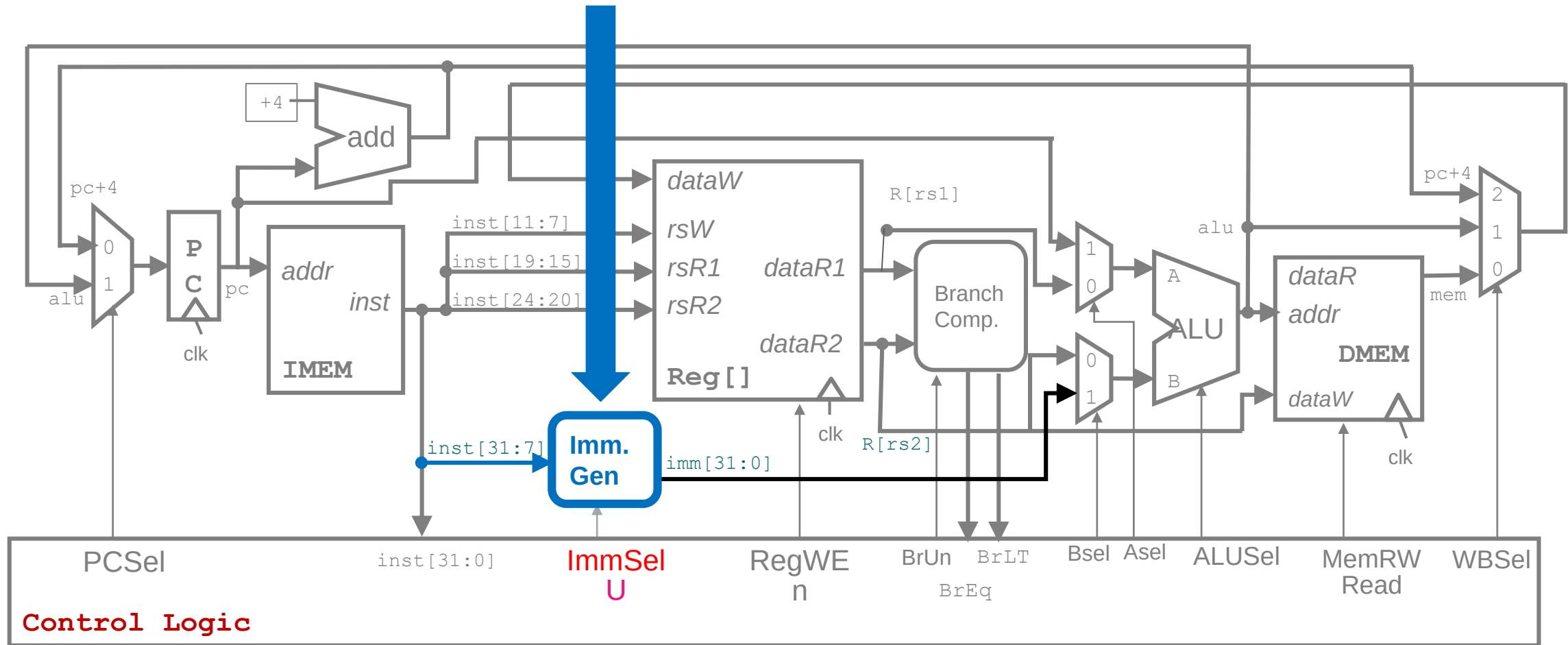
opname rd, imm



- Used for two instructions:
 - Lui** : Load Upper Immediate
 - Auipc** : Add Upper Immediate to PC
 - Both increment PC to next instruction and save to destination register.

U-Format Block update: Immediates

Generate **imm** with upper
20 bits. (U-format)



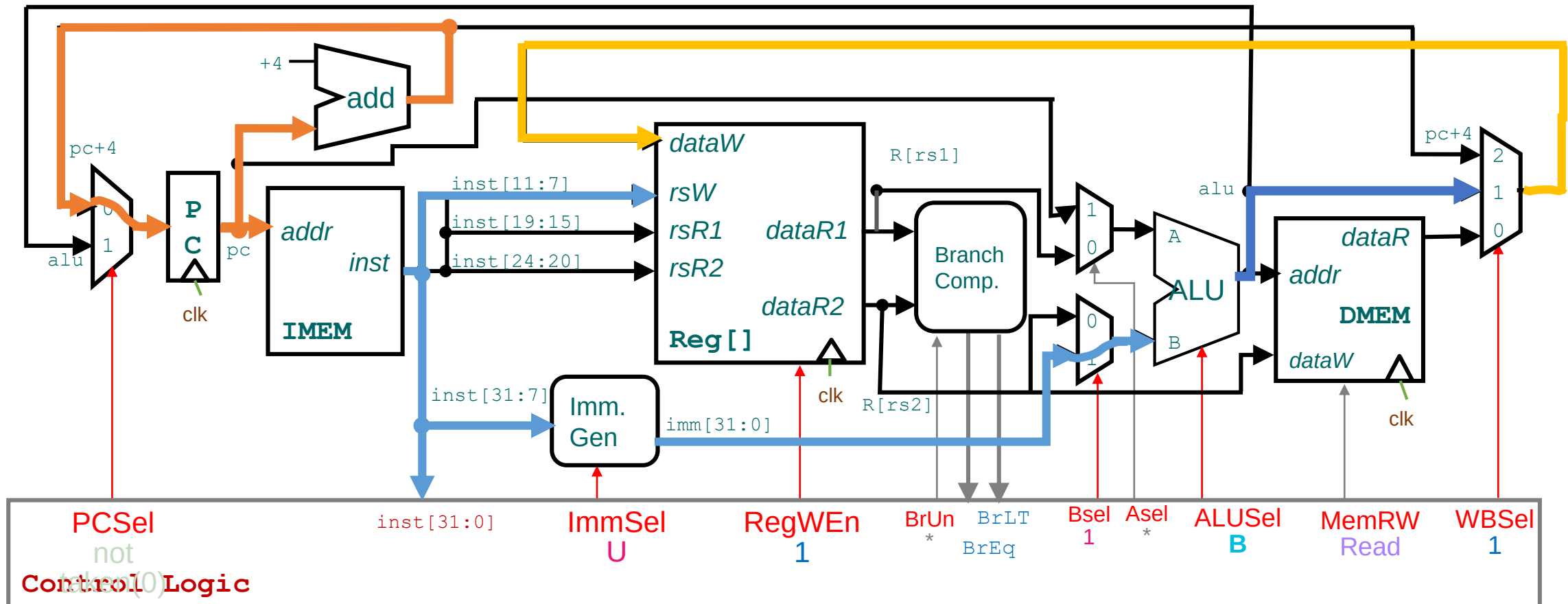
Increment PC to next instruction.

Generate imm with upper 20 bits. (U-format)

Grab only imm!
(ALUSel=B)

Don't write to memory.

Write result to destination register.



ensia AUIPC Datapath

$$PC = PC + 4$$

$$R[rd] = PC + imm$$

Increment PC to next instruction.

Generate **imm** with upper 20 bits. (U-format)

Add PC + **imm**!

Don't write to memory.

Write result to destination register.

